

Cognition on Graph: Navigating Massive Knowledge Space via Cognitive Cycles and Bidirectional Graph-Text Synergy

Gengxian Zhou^{1,2} Jian Xu^{3,4} Zichen Tang¹ Shiming Xiang^{3,4}
 Haihong E^{1,*} Cheng-Lin Liu^{3,4,2,*}

¹Beijing University of Posts and Telecommunications

²Zhongguancun Academy, Beijing

³MAIS, Institute of Automation, Chinese Academy of Sciences

⁴School of Artificial Intelligence, University of Chinese Academy of Sciences

Correspondence: ehaihong@bupt.edu.cn liucl@nlpr.ia.ac.cn

Abstract

Retrieval-Augmented Generation (RAG) has empowered Large Language Models (LLMs) to tackle knowledge-intensive tasks. However, navigating global, heterogeneous knowledge bases (large-scale knowledge graphs and text corpora) for complex reasoning remains a challenge. Existing methods typically employ reactive, graph-driven exploration strategies, which blindly follow graph topology without adapting to the question context or evolving exploration progress, and lack deep bidirectional synergy between graph and text. To address these limitations, we propose CoG (Cognition on Graph), a cognitive-inspired, training-free framework for adaptive knowledge exploration. Drawing inspiration from human problem-solving, CoG performs a continuous *plan-explore-reflect* cycle, where it proactively formulates investigation plans, performs dual-source retrieval, and dynamically reflects on progress to adjust strategies. Crucially, it establishes deep bidirectional synergy between structured graph and unstructured text, where entities extracted from text dynamically guide graph exploration to bridge knowledge gaps. Extensive experiments on seven multi-hop QA benchmarks demonstrate that CoG significantly outperforms state-of-the-art methods while achieving superior exploration efficiency. Our code and datasets are available at <https://github.com/zhougengxian/CoG>.

1 Introduction

While Large Language Models (LLMs) excel in various tasks, their reliance on static parametric knowledge causes factual hallucinations and opaque reasoning in knowledge-intensive scenarios (Huang et al., 2025b; Li et al., 2025b; Wang et al., 2026). Retrieval-Augmented Generation (RAG) mitigates this by grounding response in external knowledge sources to improve accuracy and traceability (Zhang et al., 2025b; Song et al., 2025).

*Corresponding authors.

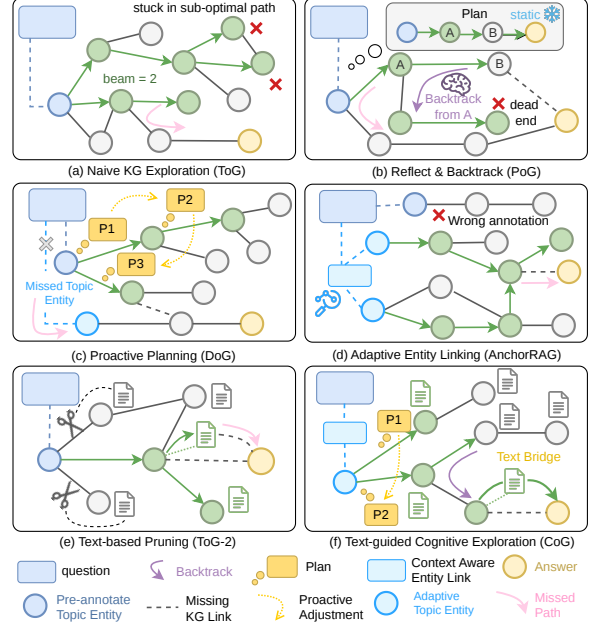


Figure 1: Comparison of exploration strategies. CoG integrates all prior capabilities in a cognitive cycle, uniquely establishing a bidirectional synergy where text actively bridges graph gaps, enabling robust navigation in global knowledge bases.

Effective retrieval is the core of RAG. Vector-RAG (Lewis et al., 2020) retrieves text chunks via vector similarity, while GraphRAG (Gutiérrez et al., 2025; Luo et al., 2025a; Gutiérrez et al., 2024; Yang et al., 2025; Wu et al., 2026; Dong et al., 2026) constructs Knowledge Graphs (KGs) for structural guidance. However, existing methods predominantly operate in simplified, local settings. They restrict the knowledge scope to pre-selected oracle paragraphs known to contain answers, rather than raw, complete entity documents. This masks the inherent noise of real-world retrieval, failing to assess true robustness (Luo et al., 2025b).

When transitioning to the global setting, the core challenge escalates from simple information retrieval to complex knowledge exploration. It requires systems to progressively construct evidence chains by synthesizing structural associations from

KGs and semantic information from text corpora. Throughout this process, the systems must effectively navigate the noise, ambiguity and distractions inherent in large-scale knowledge bases.

Existing works attempting to address this challenge, however, fall short in two critical aspects (illustrated in Figure 1). First, exploration remains heavily **KG-dependent** with **insufficient text utilization**. Most methods rely solely on KG traversal for exploration (Sun et al., 2024; Chen et al., 2024b; Ma et al., 2025a; Wang et al., 2025). Even hybrid methods (Ma et al., 2025b) merely use text as auxiliary context, failing to leverage entities within text to actively guide the exploration, leaving the inherent sparsity and incompleteness of KGs unaddressed. Second, the exploration strategies are **fragile and rigid**. Relying on pre-annotated entities as rigid starting points (Xu et al., 2026) and lacking mechanisms for backtracking (Chen et al., 2024b) or proactive adjustment (Ma et al., 2025a), these systems are prone to getting lost in irrelevant subgraphs or stranded by linking failures or knowledge gaps.

To bridge these gaps, we draw inspiration from the cognitive process of human problem-solving. Facing complex questions in open domains, humans do not blindly traverse information; instead, they engage in a dynamic cycle of planning, exploration, and reflection. They proactively formulate investigation plans, flexibly synthesize information from diverse sources, and continuously reflect on accumulated evidence to adjust their strategy.

Guided by this mechanism, we propose CoG (Cognition on Graph), a framework that instantiates this human-like reasoning process into a continuous cognitive loop comprising Planning, Exploration, Synthesis, and Reflection. In the **Planning** phase, instead of relying on pre-annotated entities, CoG proactively analyzes the question to formulate an investigation strategy, from which it derives exploratory sub-queries and identifies relevant entities to anchor the search. During the **Exploration** phase, CoG performs dual-source knowledge gathering. It retrieves 1-hop relevant subgraphs from the KG while simultaneously conducting hierarchical reading of entity-related documents in the text corpus, capturing both structural relations and rich contextual details. The **Synthesis** module then aggregates evidence from both sources into a unified retrieval summary, distilling key insights that advance the reasoning process and achieving deep bidirectional synergy between structured and un-

structured knowledge. Finally, the **Reflection** module evaluates the utility of the retrieved information: for productive explorations, it updates the strategy and generates the next query based on the progress made; for unproductive ones, it diagnoses the failure using the global interaction history and adjusts the course, ensuring robust navigation through vast knowledge spaces.

Our main contributions are threefold:

- We propose CoG, a cognitive-inspired, closed-loop, training-free GraphRAG framework. It leverages continuous cognitive cycles of plan-explore-reflect for **adaptive knowledge exploration** in noisy global knowledge bases.
- We establish deep **bidirectional synergy** between structured and unstructured knowledge, where entities dynamically extracted from text actively guide graph exploration to effectively address KG incompleteness.
- Extensive experiments on seven knowledge-intensive, multi-hop QA benchmarks demonstrate that CoG significantly outperforms state-of-the-art methods, showcasing superior reasoning capability and robustness in navigating large-scale knowledge bases.

2 Related Work

RAG enhances LLMs by retrieving external knowledge. Beyond VectorRAG (Lewis et al., 2020), GraphRAG methods construct structured indices like knowledge graphs (Gutiérrez et al., 2024; Guo et al., 2024; Gutiérrez et al., 2025; Zhu et al., 2025), hierarchical communities (Edge et al., 2024; Huang et al., 2025a), or document graphs (Wang et al., 2024; Chen et al., 2025; Li et al., 2024a) to guide retrieval. Recent advancements employ reinforcement learning (Luo et al., 2026; Yu et al., 2026) or hybrid dense retrieval (Lei et al., 2025) over graphs. However, these works primarily operate in local settings, restricting the knowledge base to highly relevant oracle documents or domain-specific KGs. This simplifies the exploration challenge, failing to assess robustness in noisy global environments. In contrast, CoG addresses the realistic and massive scale of global knowledge bases (Wikidata and Wikipedia), navigating an exponentially larger and noisier search space.

For global knowledge exploration, prior works focus on iterative KG reasoning. Path-based methods like ToG (Sun et al., 2024), StructGPT (Jiang et al., 2023), and KG-Agent (Jiang et al., 2025)

Method	Scope	Text Corpus		Knowledge Graph			Key Capabilities				Exploration Strategy
		#Words	#Ent.	Source	#Ent.	#Trip.	Text Guide	Refl. & Bk.	Pro. Plan.	Adapt. EL.	
VectorRAG	Local	1.5M	21K	—	—	—	—	—	—	—	Dense Retrieval
GraphRAG Series [†]	Local	0.9M	12K	Constr. [‡]	97K	1.4M	—	—	—	—	Graph-Guided Retrieval
StructGPT, KG-Agent	Global	—	—	Freebase	39M	1.9B	✗	✗	✗	✗	Interaction History
ToG, ReKnoS	Global	—	—	Wikidata	115M	1.7B	✗	✗	✗	✗	Beam Search
PoG	Global	—	—	Freebase	39M	1.9B	✗	✓	✗	✗	Beam Search w/ Backtrack
DoG	Global	—	—	Freebase	39M	1.9B	✗	✗	✓	✗	Multi-Agent Debate
AnchorRAG	Global	—	—	Freebase	39M	1.9B	✗	✗	✗	✓	Parallel Anchor Exploration
KERAG	Global	—	—	Wikidata	115M	1.7B	✗	✗	✗	✓	Schema-Guided Expansion
ToG-2	Global	5B	7.1M	Wikidata	115M	1.7B	✗	✗	✗	✗	Beam Search
CoG (Ours)	Global	5B	7.1M	Wikidata	115M	1.7B	✓	✓	✓	✓	Cognitive Cycle

Table 1: Comparison of retrieval and exploration methods. We report knowledge base scales as total **Words** and **Entity** documents for Text, versus **Entities** and **Triples** for KG (Units: K=Thousand, M=Million, B=Billion). **CoG** uniquely operates on a massive scale while integrating all key exploration capabilities: **Text Guide** (Text entity guides graph exploration), **Refl. & Bk.** (Reflection & Backtrack), **Pro. Plan.** (Proactive planning), and **Adapt. EL.** (Adaptive entity linking). [†]: Includes HippoRAG, HippoRAG2, GFM-RAG, et al. [‡]: Graphs constructed from local text corpora. ✓: Present, ✗: Absent, “—”: not applicable.

perform prompt-driven exploration, while ReKnoS (Wang et al., 2025) and KERAG (Sun et al., 2025) improve coverage via super-relations or schemas. Strategy-based methods introduce specific heuristics, such as backtracking in PoG (Chen et al., 2024b) or multi-agent debate and parallelism in DoG (Ma et al., 2025a) and AnchorRAG (Xu et al., 2026). However, these approaches treat advanced reasoning behaviors as fragmented additions to fixed pipelines. Meanwhile, learning-based methods (Chang et al., 2026; Lin et al., 2026; Cui et al., 2025) train specialized navigation modules, introducing high training costs and instability while lacking explicit interpretability. Distinguishing itself from these rigid pipelines and costly training, CoG introduces a training-free, closed-loop cognitive cycle. It natively unifies “plan-explore-reflect”, enabling proactive planning and adaptive error-correction to emerge autonomously.

Despite advanced reasoning, KG-centric methods remain vulnerable to KG sparsity and incompleteness. To address this, hybrid methods integrate KGs with text corpora (Lee et al., 2025; Sarmah et al., 2024; Tan et al., 2025). CoK (Li et al., 2024b) verifies reasoning across multi-source bases, while ToG-2 (Ma et al., 2025b) uses graph entities to anchor document retrieval. Other approaches utilize text merely for relevance scoring (Lei et al., 2025) or rely on pre-constructed hyperedges (Luo et al., 2026). However, these methods treat text as passive context or require rigid offline constructions, lacking dynamic structural guidance. In contrast, CoG introduces a deep bidirectional synergy where unstructured text actively guides graph traversal. By extracting entities from text to dynamically bridge

KG structural gaps, it achieves robust exploration in noisy global spaces.

Table 1 provides a systematic comparison of these methods across key capabilities.

3 Method

3.1 Task Definition

We focus on the task of multi-hop Question Answering (QA) over a global, hybrid knowledge base. Given a complex question q , the system must construct an evidence chain by iteratively retrieving and reasoning over heterogeneous knowledge sources to generate the final answer a .

The knowledge base $\mathcal{K} = \{\mathcal{G}, \mathcal{D}\}$ comprises two complementary components:

Structured Knowledge Graph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, where $\mathcal{E}$ and $\mathcal{R}$ denote the sets of entities and relations. The KG provides structured information in two forms: 1) relational fact triples $\mathcal{T} = \{(h, r, t) \mid h, t \in \mathcal{E}, r \in \mathcal{R}\}$ that connect pairs of entities, and 2) entity attributes (e.g., date of birth, gender) that describe the properties of individual entities.

Unstructured Text Corpus $\mathcal{D}$, a large-scale collection of documents. Each document $d \in \mathcal{D}$ corresponds to an entity and is hierarchically structured, containing a summary paragraph, a table of contents, and the full text organized into sections. It provides rich, fine-grained semantic descriptions that complement the factual statements in $\mathcal{G}$.

3.2 CoG Framework

Figure 2 illustrates the overall architecture of CoG. Given a question q , CoG iteratively executes a cognitive cycle until sufficient evidence is gathered to generate an answer. Each cycle consists

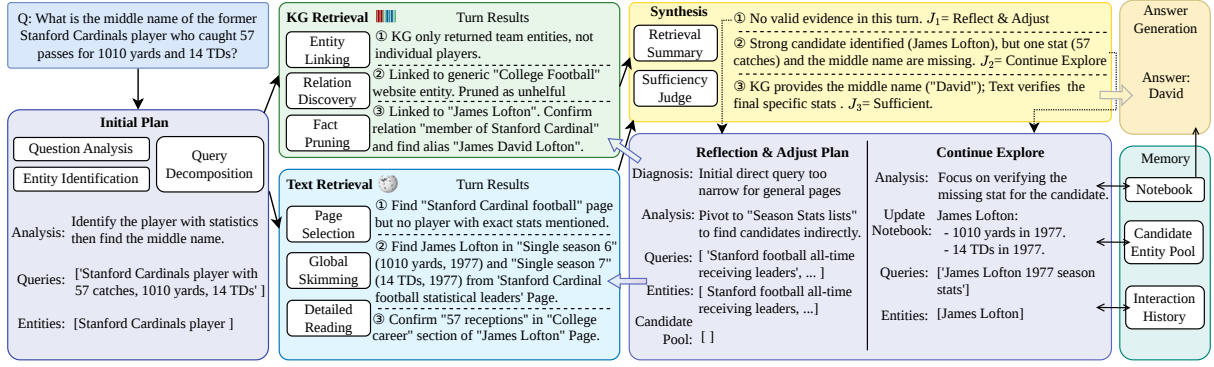


Figure 2: Overview of the CoG framework. It employs a continuous cognitive cycle of plan-explore-reflect to adaptively navigate structured and unstructured knowledge for complex QA.

of four tightly integrated phases: (1) Planning formulates an investigation strategy and generates a sub-query; (2) Exploration retrieves evidence from both KG and text; (3) Synthesis aggregates and distills the retrieved information; (4) Reflection evaluates progress and decides the next action.

Throughout this process, CoG maintains three long-term memory components to enable coherent multi-turn reasoning: a *Notebook* $\mathcal{N}$ to accumulate verified facts, a *Candidate Entity Pool* $\mathcal{P}$ to manage promising but deferred exploration leads, and an *Interaction History* $\mathcal{H}$ to enable global reflection and strategic adjustment.

3.2.1 Planning

The Planning Module proactively analyzes the question and formulates an initial exploration strategy. Unlike methods that rely on pre-annotated entities, CoG dynamically derives a strategic plan directly from the question q . This process comprises three key components:

Question Analysis. It analyzes the question q to identify information needs and reasoning dependencies, determining what knowledge must be gathered first. This analysis A_0 serves as a high-level guide for subsequent actions.

Query Decomposition. Based on the analysis, the question is decomposed into a set of focused sub-queries $Q_0 = \{q_1^{(0)}, \dots, q_B^{(0)}\}$, each targeting a specific information need that can be explored independently as the immediate next step.

Entity Identification. It identifies a list of anchor entities $E_0 = \{e_1^{(0)}, \dots, e_B^{(0)}\}$ corresponding one-to-one with Q_0 . Each $e_i^{(0)}$ serves as the core anchor point for initiating the exploration of query $q_i^{(0)}$ in both the KG and text corpus.

Formally, the initial plan P_0 is generated by:

$$P_0 = (A_0, Q_0, E_0) = \text{Prompt}_{\text{init}}(q, \text{LLM}). \quad (1)$$

The prompt is provided in Appendix K, Table 16.

3.2.2 Exploration

Given a sub-query $q_i^{(t)}$ and its anchor entity $e_i^{(t)}$ from the Planning phase, the Exploration module performs **dual-source retrieval** to gather complementary evidence from both the structured KG $\mathcal{G}$ and the unstructured text corpus $\mathcal{D}$.

KG Exploration. To navigate massive global KGs with inherent ambiguity and noise, we design a three-stage pipeline that progressively narrows the search scope from entity disambiguation to relation filtering and fine-grained fact pruning.

(1) Entity Linking. We design a context-aware coarse-to-fine recall strategy. First, we retrieve top- k candidate entities $\mathcal{C}$ via vector similarity between the query context $C_i^{(t)} = (q, A_t, q_i^{(t)}, e_i^{(t)})$ and entity profiles (aliases, description and neighbors). Then, the LLM performs precise disambiguation by evaluating each candidate’s profile against the query context to determine the optimal match:

$$\begin{aligned} \mathcal{C} &= \text{VectorRecall}(C_i^{(t)}, \mathcal{E}, k), \\ e^* &= \text{Prompt}_{\text{link}}(C_i^{(t)}, \mathcal{C}, \text{LLM}). \end{aligned} \quad (2)$$

(2) Relation Discovery. First, we retrieve all relations R_e connected to the linked entity e^* . Then, the LLM selects a subset $R_{\text{sel}} \subset R_e$ relevant to the query context $C_i^{(t)}$ by considering relation labels, connectivity and potential to lead toward the answer, thereby pruning the search space at the schema level:

$$\begin{aligned} R_{e^*} &= \text{GetRelations}(e^*, \mathcal{G}), \\ R_{\text{sel}} &= \text{Prompt}_{\text{rel}}(C_i^{(t)}, e^*, R_{e^*}, \text{LLM}). \end{aligned} \quad (3)$$

(3) Fact Pruning. We retrieve all facts F_{raw} (triples and attributes) associated with the selected

relations R_{sel} . The LLM performs fine-grained pruning to retain only facts that either directly answer the query $q_i^{(t)}$ or serve as promising stepping stones for further exploration, filtering out noise while preserving critical evidence:

$$\begin{aligned} F_{\text{raw}} &= \{(e^*, r, t) \in \mathcal{T} \mid r \in R_{\text{sel}}, t \in \mathcal{E}\} \\ &\quad \cup \{(h, r, e^*) \in \mathcal{T} \mid r \in R_{\text{sel}}, h \in \mathcal{E}\}, \\ F_i^{KG} &= \text{Prompt}_{\text{fact}}(C_i^{(t)}, F_{\text{raw}}, \text{LLM}). \end{aligned} \quad (4)$$

Text Exploration. We propose a hierarchical reading strategy that mimics human reading behavior, narrowing the focus from document retrieval to global skimming and fine-grained section analysis, balancing retrieval efficiency with depth.

(1) Adaptive Page Selection. We employ an adaptive mechanism to retrieve the optimal entity document d_i for $e_i^{(t)}$. When encountering disambiguation or missing pages, the LLM progressively refines the search query by analyzing query context $C_i^{(t)}$ and Wikipedia API feedback, ensuring robust page localization despite real-world noise.

(2) Global Skimming. The LLM skims the high-level structure of the retrieved page d_i (summary, table of contents and infobox) to assess the page’s relevance to the query context $C_i^{(t)}$ and extract useful evidence I_{skim} . If deeper investigation is needed, it selects a focused subset of promising sections S_{sel} likely to contain missing details:

$$S_{\text{sel}}, I_{\text{skim}} = \text{Prompt}_{\text{skim}}(C_i^{(t)}, d_i, \text{LLM}). \quad (5)$$

(3) Detailed Reading. For each selected section $s \in S_{\text{sel}}$, we retrieve its full content, split it into chunks, and identify the top- k relevant chunks via dense retrieval. Concurrently, tables within the section are extracted and serialized. Finally, the LLM performs joint analysis of text chunks and tables to synthesize fine-grained evidence I_{detail}^s :

$$I_{\text{detail}}^{(s)} = \text{Prompt}_{\text{detail}}(C_i^{(t)}, s, \text{LLM}). \quad (6)$$

Combining skimmed and detailed extractions, the final text evidence for entity $e_i^{(t)}$ is $F_i^{\text{text}} = I_{\text{skim}} \cup \{I_{\text{detail}}^{(s)} \mid s \in S_{\text{sel}}\}$.

Dual-Source Evidence. The final retrieved evidence for sub-query $q_i^{(t)}$ is the aggregation of dual-source knowledge: $K_i^{(t)} = F_i^{KG} \cup F_i^{\text{text}}$.

Prompts for KG (Tables 17–20) and text (Tables 21–24) exploration are provided in Appendix K.

3.2.3 Synthesis & Reflection

The **Synthesis** module aggregates dual-source evidence and evaluates progress. We design structured formats to present KG and text retrieval $K_t = \{K_1^{(t)}, \dots, K_B^{(t)}\}$. The LLM first distills key facts and promising leads into a unified retrieval summary M_t , achieving deep synergy between structured KG and unstructured text. Simultaneously, it makes a judgment J_t based on the notebook $\mathcal{N}$ and the current plan $P_t = (A_t, Q_t, E_t)$ to determine whether to generate the final answer, continue exploration, or adjust the strategy:

$$J_t, M_t = \text{Prompt}_{\text{syn}}(q, P_t, K_t, \mathcal{N}, \text{LLM}). \quad (7)$$

Depending on J_t , the **Reflection** module dynamically directs exploration through two pathways:

(1) Continue Exploration. When progress is made but evidence remains incomplete, it refines the analysis to reflect new findings, formulates a new plan P_{t+1} targeting remaining knowledge gaps. Simultaneously, it updates two long-term memory structures: Notebook $\mathcal{N}$, which accumulates verified key facts directly relevant to q , and Candidate Entity Pool $\mathcal{P}$, which stores promising but deferred entities due to exploration width constraints:

$$P_{t+1}, \mathcal{N}, \mathcal{P} = \text{Prompt}_{\text{cont}}(q, P_t, M_t, \mathcal{N}, \mathcal{P}, \text{LLM}). \quad (8)$$

(2) Strategy Adjustment. When facing dead ends, it performs critical reflection by analyzing the global interaction history $\mathcal{H}$ to diagnose strategic errors. Then it pivots the strategy by backtracking to deferred entities in $\mathcal{P}$ or reformulating queries from entirely new angles based on the q and current progress, thereby ensuring resilient exploration:

$$P_{t+1}, \mathcal{P} = \text{Prompt}_{\text{recov}}(q, P_t, M_t, \mathcal{H}, \mathcal{N}, \mathcal{P}, \text{LLM}). \quad (9)$$

Interaction History Update. After each turn, the system appends the current retrieval summary M_t and plan P_t to the interaction history: $\mathcal{H} \leftarrow \mathcal{H} \cup \{(P_t, M_t)\}$. This cumulative memory enables global retrospection, allowing the system to avoid repeating failed strategies and to make globally informed decisions across multiple turns.

Prompts for synthesis (Table 25) and reflection (Tables 28, 29) are provided in Appendix K.

3.2.4 Overall Workflow & Answer Generation

CoG executes the aforementioned cognitive cycle iteratively, until the Reflection module deems the evidence sufficient or a maximum turn limit is

reached. The complete algorithmic workflow is detailed in Algorithm 1 in Appendix A.

Upon sufficiency, the system synthesizes accumulated facts in Notebook $\mathcal{N}$ and final retrieval result M_t . The LLM constructs a comprehensive reasoning chain connecting these facts to directly address the question, producing the final answer a .

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate on seven knowledge-intensive multi-hop QA benchmarks: (1) KG-based QA: KGQAGen (Zhang et al., 2025a), CWQ (Talmor and Berant, 2018), QALD10-en (Usbeck et al., 2024), and WebQSP (Yih et al., 2016), which require reasoning over structured facts for entity-centric answers. (2) Text-based QA: 2WikiMQA (Ho et al., 2020), AdvHotpotQA (Ye and Durrett, 2022), and MusiQue (Trivedi et al., 2022), which demand deep semantic understanding and multi-document synthesis for span extraction and free-form answer generation. Detailed descriptions and statistics are provided in Appendix B. We use Exact Match (EM) as the primary evaluation metric following previous work (Ma et al., 2025b), and additionally report token-level F1 and reference-based LLM-as-judge results in Appendix D. For repeated runs, we report the median of three independent runs to ensure robustness.

Baselines. We compare CoG against representative methods across four categories: (1) LLM-only: Direct few-shot prompting, Chain-of-Thought (Wei et al., 2022), and Self-Consistency (Wang et al., 2023), relying solely on internal parametric knowledge; (2) Text-based RAG: VectorRAG (top-3 chunk dense retrieval), ReAct (Yao et al., 2023) and IRCoT (Trivedi et al., 2023) for iterative text exploration, and Search-o1 (Li et al., 2025a) for on-demand retrieval during reasoning; (3) KG-based RAG: ToG (Sun et al., 2024) for iterative KG traversal, PoG (Chen et al., 2024b) with backtracking, and DoG (Ma et al., 2025a) using debate-driven query evolution; (4) Hybrid RAG: ToG-2 (Ma et al., 2025b), the current state-of-the-art framework combining KG traversal with text retrieval under the same global knowledge setting as CoG.

Implementation Details. Following previous works (Ma et al., 2025b; Li et al., 2024b), we adopt the global knowledge setting, utilizing the full Wikidata and Wikipedia as structured and un-

structured knowledge bases, respectively (details in Appendix C). This setting presents a realistic and challenging environment compared to simplified local settings. We employ Qwen3-32B (non-thinking mode) as the backbone LLM for CoG and baselines to ensure fair comparison. For retrieval, we utilize bge-m3 (Chen et al., 2024a) as the embedding model for text chunk retrieval, and Qwen3-Embedding-4B (Zhang et al., 2025c) for entity linking in the KG. We set the maximum exploration depth to 4 and width (sub-queries per turn) to 5. For entity linking, we retrieve the top-20 candidate entities. In text exploration, we extract up to 3 relevant chunks per section.

4.2 Main Results

As shown in Table 2, CoG substantially outperforms baselines across most benchmarks, with advantages particularly pronounced on complex tasks. On the challenging text-heavy MusiQue, CoG (27.8%) significantly surpasses the strongest text agent IRCoT (20.6%). Similarly, it achieves a 34.3% relative gain over ToG-2 on the high-quality KG-based KGQAGen. Meanwhile, CoG maintains comparable performance on simpler WebQSP (1-2 hops). These results validate our cognitive framework’s effectiveness in unifying structured and unstructured knowledge exploration. Additional F1 and LLM-as-judge results in Appendix D confirm that these gains persist under both lexical-overlap and semantic evaluation.

Analysis of Baselines VectorRAG struggles with KG-based tasks, as its flat retrieval lacks structural guidance. Agentic text baselines (ReAct, IRCoT, Search-o1) and strong KG agents (PoG, DoG) improve over basic RAG but exhibit single-source limitations: text agents falter on KG-heavy tasks, while DoG struggles on text-heavy ones. ToG-2 effectively combines KG and text to outperform single-source baselines, but its reactive exploration lacks long-term planning and bidirectional synergy, faltering on complex tasks. CoG consistently outperforms single-source and hybrid baselines, proving that our Graph-Text Synergy effectively overcomes pure graph limitations and text retrieval noise, yielding benefits beyond a generic agentic loop.

4.3 Ablation Study

To dissect CoG’s components, we conduct ablation studies across three dimensions (Table 3).

Baseline Type	Method	Multi-hop KG-based QA				Multi-hop text-based QA		
		KGQAGen	CWQ	QALD10-en	WebQSP	2WikiMQA	AdvHotpotQA	MusiQue
LLM-only	Direct	36.70	32.42	47.45	72.45	47.20	20.13	8.40
	CoT	38.28	33.59	45.35	70.84	47.40	23.70	9.60
	CoT-SC	40.50	33.40	46.55	71.40	49.60	24.03	10.60
Text-based RAG	Vector RAG	48.66	30.76	39.04	70.49	52.60	32.14	12.40
	ReAct	50.79	34.18	41.44	76.15	63.20	41.88	17.00
	IRCoT	53.75	33.30	41.14	71.40	71.60	47.08	20.60
	Search-o1	46.34	36.72	43.54	72.03	44.60	40.26	19.40
KG-based RAG	ToG	46.90	28.42	48.65	73.28	50.40	18.83	9.20
	PoG	50.93	27.15	48.95	78.53	73.20	15.26	14.00
	DoG	54.77	33.11	55.86	82.38	61.20	20.13	11.40
Hybrid RAG	ToG-2	55.61	41.41	53.45	81.89	73.00	35.71	16.20
Proposed	CoG	74.70	42.87	55.26	81.75	85.20	51.95	27.80

Table 2: Main results (Exact Match, %) of different methods on seven multi-hop QA datasets. All methods use Qwen3-32B (non-thinking mode) as the backbone LLM. The best results are highlighted in **bold**.

Variant	KGQA	CWQ	QALD	2Wiki	AdvHot	MusiQ	Avg.
CoG (Full)	74.70	42.87	55.26	85.20	51.95	27.80	56.30
<i>Impact of Knowledge Sources</i>							
w/o Text	75.90	39.06	51.65	76.20	27.27	20.20	48.38
w/o KG	56.44	40.92	48.65	79.80	49.68	20.80	49.38
<i>Impact of Bidirectional Synergy</i>							
w/o T-guide	74.14	42.58	52.85	76.20	39.61	22.80	51.36
w/o Synthesis	73.12	41.11	50.15	83.60	43.83	24.20	52.67
<i>Impact of Cognitive Strategy</i>							
w/o Reflect	72.57	42.38	52.85	85.00	48.05	25.80	54.44
w/o Plan	78.78	43.85	50.75	81.00	47.73	25.40	54.59
w/o Adap.EL	70.81	44.14	53.15	86.00	47.73	23.40	54.21

Table 3: Ablation results (EM, %) of CoG across three groups: (1) **Knowledge Sources**: removing either KG (*w/o KG*) or Text (*w/o Text*); (2) **Bidirectional Synergy**: disabling text-guided KG exploration (*w/o T-guide*), or removing summary synthesis module (*w/o Synthesis*); (3) **Cognitive Strategy**: excluding reflection-based adjustment (*w/o Reflect*), proactive planning (*w/o Plan*), or adaptive entity linking (*w/o Adap.EL*).

Knowledge Sources. Removing either source causes significant drops (-6.92% for *w/o KG*, -7.92% for *w/o Text*). The severe degradation of *w/o Text* on text-heavy datasets (AdvHotpotQA, MusiQue) confirms that while KG provides structural guidance, rich text semantics are indispensable for bridging knowledge gaps.

Bidirectional Synergy. Simply accessing both sources is insufficient. Disabling text-to-KG feedback (*w/o T-guide*) drops performance by 4.94%, indicating that entities discovered from text serve as effective new anchors for bridging disconnected KG paths. Furthermore, removing the synthesis module (*w/o Synthesis*) degrades performance, indicating that explicitly distilling heterogeneous evidence is crucial for filtering noise and informing

subsequent reasoning.

Cognitive Strategy. Cognitive mechanisms ensure robust exploration. Removing reflection, planning, or adaptive entity linking all degrades average performance. These results suggest that CoG benefits not only from access to KG and text, but also from its closed-loop control: reflection helps recover from noisy paths, planning guides goal-oriented search, and adaptive entity linking provides reliable anchors under ambiguous mentions.

4.4 In-depth Analysis

Generalizability and Cost-Effectiveness. Table 4 demonstrates CoG’s effectiveness across diverse LLMs (detailed in Appendix E). Notably, CoG unlocks the reasoning potential of smaller models, yielding a massive $+113.2\%$ gain on Qwen3-8B. Remarkably, CoG-8B (51.0%) outperforms all strong 32B baselines (e.g., IRCoT 44.6%, ToG-2 45.9%). This confirms that CoG’s superiority stems from its robust cognitive architecture rather than mere parameter scale, effectively mitigating error propagation on smaller LLMs while offering a highly cost-effective solution.

Impact of Exploration Depth. Figure 3 illustrates the performance trends across varying depths (see Appendix F for full results). On KG tasks, CoG consistently outperforms VectorRAG across all depths, confirming the superiority of structured cognitive exploration over flat retrieval. Similarly, text tasks show steady gains as depth increases, indicating that deeper exploration effectively gathers scattered semantic evidence. Notably, the optimal depth correlates strongly with dataset difficulty.

Backbone	Method	KG Avg.	Text Avg.	Overall Avg.
Qwen3-32B	Direct	41.48	25.24	33.36
	CoG	59.31	54.98	57.15 (+71.3%)
Qwen3-8B	Direct	31.09	16.73	23.91
	CoG	53.39	48.55	50.97 (+113.2%)
gpt-oss-120b	Direct	28.33	15.88	22.10
	CoG	42.87	40.28	41.58 (+88.1%)
DeepSeek-V3.2	Direct	47.91	28.06	37.99
	CoG	64.79	60.50	62.64 (+64.9%)
GPT-5.1	Direct	45.67	38.67	42.17
	CoG	56.00	55.33	55.67 (+32.0%)

Table 4: Performance comparison (EM, %) of CoG versus Direct Prompting across different backbone LLMs. Proprietary models are evaluated on a sampled subset (100 samples per dataset) due to cost constraints.

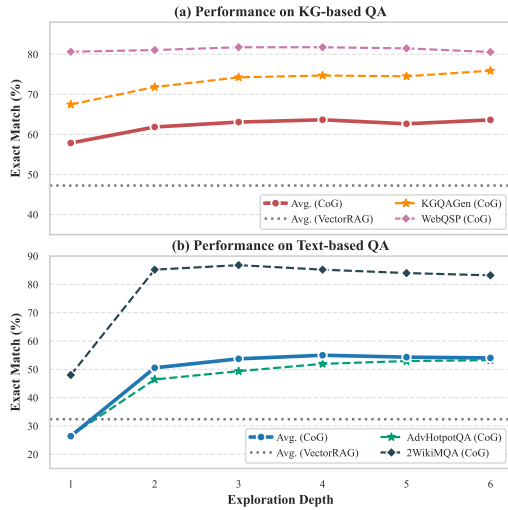


Figure 3: Impact of exploration depth on KG-based and Text-based tasks and representative datasets.

Simpler datasets (WebQSP, 2WikiMQA) achieve optimal performance at shallower depths and then stabilize, whereas challenging ones (KGQAGen, AdvHotpotQA) continue to benefit from deeper exploration. It demonstrates CoG’s adaptability to varying task complexity.

Exploration Efficiency. We evaluate efficiency across four dimensions: accessed documents, token consumption, latency, and wall-clock time per retrieval round (detailed breakdown in Appendix G). Despite multiple LLM calls per turn, CoG’s goal-oriented planning achieves remarkable precision, accessing $12.4\times$ fewer documents than ToG-2’s exhaustive beam search (Table 5). Regarding absolute cost, CoG exhibits complexity adaptability: on simpler datasets (e.g., WebQSP), it maintains low token usage ($\sim 15k$) comparable to text agents like IRCOT ($\sim 22k$) while achieving superior accuracy. On complex tasks (e.g., MusiQue), costs increase to support deep reasoning, trading computation for

Method	KGQA	CWQ	QALD	WebQ	2Wiki	Hotpot	MusiQ	Avg.
ToG-2	37.1	40.4	31.2	22.9	17.7	36.1	42.2	32.5
CoG	3.1	3.1	2.3	1.5	2.4	2.5	3.5	2.6
Gain	$\times 11.9$	$\times 13.2$	$\times 13.3$	$\times 15.5$	$\times 7.3$	$\times 14.8$	$\times 12.1$	$\times 12.4$

Table 5: Comparison of exploration efficiency (Average number of unique documents accessed per question).

capability where efficient methods fail. Crucially, this overhead can be mitigated by using smaller models, as CoG-8B already outperforms 32B baselines (Appendix E), improving the accuracy-cost trade-off for deployment.

Hyperparameter Robustness. We study the sensitivity of adaptive entity linking by varying the retrieved entity candidates from $k = 6$ to 30 (Appendix H). CoG remains stable across settings, with average EM varying only slightly from 59.1% to 60.3%. Increasing k does not degrade performance, suggesting that CoG’s context-aware disambiguation can filter noisy candidates while benefiting from broader entity recall.

Manual Evaluation. We manually analyze sampled reasoning paths in Appendix I to evaluate reasoning faithfulness and diagnose error causes. For EM-correct cases, CoG yields the highest proportion of *True Positives* (89.1%) with faithful reasoning chains. In contrast, CoT-SC and ToG-2 suffer more severe false positives (*Weak FP + FP*: 29.1% and 12.7% vs. CoG’s 3.6%), often reaching correct answers via hallucinated or flawed intermediate steps. For EM-incorrect cases, 71.1% of CoG’s errors are attributable to external factors such as metric false negatives, dataset flaws, or temporal mismatch, while 28.9% reflect internal exploration or reasoning failures. These findings support both the reliability of CoG’s EM gains and the need for stronger exploration focus and disambiguation.

Qualitative Analysis. We provide a detailed case study in Appendix J, demonstrating how CoG’s cognitive cycle and bidirectional synergy enable it to recover from dead ends and discover critical text evidence to solve complex multi-hop queries.

5 Conclusion

In this paper, we introduced CoG, a cognitive-inspired framework for autonomous knowledge exploration in global, hybrid knowledge bases. Under the continuous “plan-explore-reflect” human cognitive cycle, CoG shifts the paradigm from passive, reactive retrieval to proactive knowledge discovery,

enabling adaptive strategy adjustment and robust recovery from exploration failures. Crucially, it establishes a deep bidirectional synergy between structured KGs and unstructured text, leveraging their complementary strengths to bridge knowledge gaps. Extensive experiments demonstrate that CoG significantly outperforms state-of-the-art baselines on multiple multi-hop QA tasks while achieving superior exploration efficiency.

Limitations

While CoG demonstrates superior performance and exploration efficiency in global knowledge exploration, several limitations remain.

Latency and Cost. The iterative cognitive loop inevitably incurs higher inference latency and token cost than passive retrieval methods. Although CoG reduces redundant document access through goal-oriented exploration and can improve the cost-accuracy trade-off when paired with smaller backbones, its multi-call workflow may still limit real-time deployment.

Dependency on Backbone LLM. The framework’s effectiveness still depends on the backbone LLM’s instruction following and long-context reasoning abilities. Although CoG substantially improves smaller models, weaker LLMs may still struggle with complex planning, evidence synthesis, or reflection over noisy intermediate results.

Acknowledgments

This work is supported by the Zhongguancun Academy (Grant No. 02012501), in part by the National Natural Science Foundation of China (Grant Nos. 62473271, 62506357), and the Fundamental Research Funds for the Beijing University of Posts and Telecommunications (Grant No. 2025AI4S03).

References

- Ge Chang, Jinbo Su, Jiacheng Liu, Pengfei Yang, Yuhao Shang, Huiwen Zheng, Hongli Ma, Yan Liang, Yuanchun Li, and Yunxin Liu. 2026. [Enhancing agentic textual graph retrieval with synthetic stepwise supervision](#). In *ACL*, pages 25510–25525, San Diego, California, United States. Association for Computational Linguistics.
- Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024a. [M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation](#). In *Findings of ACL*, pages 2318–2335, Bangkok, Thailand. Association for Computational Linguistics.
- Liyi Chen, Panrong Tong, Zhongming Jin, Ying Sun, Jieping Ye, and Hui Xiong. 2024b. [Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs](#). In *NeurIPS*, volume 37, pages 37665–37691. Curran Associates, Inc.
- Weijie Chen, Ting Bai, Jinbo Su, Jian Luan, Wei Liu, and Chuan Shi. 2025. [Kg-retriever: Efficient knowledge indexing for retrieval-augmented large language models](#). In *2025 IEEE International Conference on Knowledge Graph (ICKG)*, pages 27–34.
- Yuanning Cui, Zequn Sun, Wei Hu, and Zhangjie Fu. 2025. [Kgfr: A foundation retriever for generalized knowledge graph question answering](#). *Preprint*, arXiv:2511.04093.
- Junnan Dong, Siyu An, Yifei Yu, QIANWEN ZHANG, Linhao Luo, Xiao Huang, di yin, Yunsheng Wu, and Xing Sun. 2026. [Youtu-graphrag: Vertically unified agents for graph retrieval-augmented complex reasoning](#). In *ICLR*, volume 2026, pages 143735–143755.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. [From local to global: A graph rag approach to query-focused summarization](#). *Preprint*, arXiv:2404.16130.
- Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. [Lightrag: Simple and fast retrieval-augmented generation](#). *Preprint*, arXiv:2410.05779.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. [From RAG to memory: Non-parametric continual learning for large language models](#). In *ICML*, volume 267 of *Proceedings of Machine Learning Research*, pages 21497–21515. PMLR.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. [HippoRAG: Neurobiologically inspired long-term memory for large language models](#). In *NeurIPS*, volume 37, pages 59532–59569. Curran Associates, Inc.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. [Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps](#). In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain (Online). International Committee on Computational Linguistics.
- Haoyu Huang, Yongfeng Huang, Yang Junjie, Zhenyu Pan, Yongqiang Chen, Kaili Ma, Hongzhi Chen, and James Cheng. 2025a. [Retrieval-augmented generation with hierarchical knowledge](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 6044–6060, Suzhou, China. Association for Computational Linguistics.

- Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025b. [A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions](#). *ACM Trans. Inf. Syst.*, 43(2).
- Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. 2023. [StructGPT: A general framework for large language model to reason over structured data](#). In *EMNLP*, pages 9237–9251, Singapore. Association for Computational Linguistics.
- Jinhao Jiang, Kun Zhou, Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. 2025. [KG-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph](#). In *Proceedings of the 63rd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 9505–9523. Association for Computational Linguistics.
- Meng-Chieh Lee, Qi Zhu, Costas Mavromatis, Zhen Han, Soji Adeshina, Vassilis N. Ioannidis, Huzefa Rangwala, and Christos Faloutsos. 2025. [HybGRAG: Hybrid retrieval-augmented generation on textual and relational knowledge bases](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 879–893. Association for Computational Linguistics.
- Yongjia Lei, Haoyu Han, Ryan A. Rossi, Franck Dernoncourt, Nedim Lipka, Mahantesh M Halappanavar, Jiliang Tang, and Yu Wang. 2025. [Mixture of structural-and-textual retrieval over text-rich graph knowledge bases](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 18306–18321, Vienna, Austria. Association for Computational Linguistics.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). In *NeurIPS*, volume 33, pages 9459–9474. Curran Associates, Inc.
- Shilong Li, Yancheng He, Hangyu Guo, Xingyuan Bu, Ge Bai, Jie Liu, Jiaheng Liu, Xingwei Qu, Yangguang Li, Wanli Ouyang, Wenbo Su, and Bo Zheng. 2024a. [GraphReader: Building graph-based agent to enhance long-context abilities of large language models](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 12758–12786. Association for Computational Linguistics.
- Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. 2025a. [Search-o1: Agentic search-enhanced large reasoning models](#). In *EMNLP*, pages 5420–5438, Suzhou, China. Association for Computational Linguistics.
- Xingxuan Li, Ruochen Zhao, Yew Ken Chia, Bosheng Ding, Shafiq Joty, Soujanya Poria, and Lidong Bing. 2024b. [Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources](#). In *ICLR*, volume 2024, pages 9565–9587.
- Yangning Li, Weizhi Zhang, Yuyao Yang, Wei-Chieh Huang, Yaozu Wu, Junyu Luo, Yuanchen Bei, Henry Peng Zou, Xiao Luo, Yusheng Zhao, Chunkit Chan, Yankai Chen, Zhongfen Deng, Yinghui Li, Hai-Tao Zheng, Dongyuan Li, Renhe Jiang, Ming Zhang, Yangqiu Song, and Philip S. Yu. 2025b. [Towards agentic rag with deep reasoning: A survey of rag-reasoning systems in llms](#). *Preprint*, arXiv:2507.09477.
- Junhong Lin, Shicheng Liu, Jinyeop Song, Song Wang, Julian Shun, and Yada Zhu. 2026. [Efficient and transferable agentic knowledge graph rag via reinforcement learning](#). *Preprint*, arXiv:2509.26383.
- Haoran Luo, Haihong E, Guanting Chen, Qika Lin, Yikai Guo, Fangzhi Xu, Zemin Kuang, Meina Song, Xiaobao Wu, Yifan Zhu, and Luu Anh Tuan. 2026. [Graph-r1: Towards agentic graphrag framework via end-to-end reinforcement learning](#). *Preprint*, arXiv:2507.21892.
- Linhao Luo, Zicheng Zhao, Reza Haffari, Dinh Phung, Chen Gong, and Shirui Pan. 2025a. [Gfm-rag: Graph foundation model for retrieval augmented generation](#). In *NeurIPS*, volume 38, Main Conference, pages 36371–36405. Curran Associates, Inc.
- Qi Luo, Xiaonan Li, Tingshuo Fan, Xinchu Chen, and Xipeng Qiu. 2025b. [Towards global retrieval augmented generation: A benchmark for corpus-level reasoning](#). *Preprint*, arXiv:2510.26205.
- Jie Ma, Zhitao Gao, Qi Chai, Wangchun Sun, Pinghui Wang, Hongbin Pei, Jing Tao, Lingyun Song, Jun Liu, Chen Zhang, and Lizhen Cui. 2025a. [Debate on graph: A flexible and reliable reasoning framework for large language models](#). In *AAAI*, volume 39, pages 24768–24776.
- Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2025b. [Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation](#). In *ICLR*, volume 2025, pages 52782–52806.
- Bhaskarjit Sarmah, Dhagash Mehta, Benika Hall, Rohan Rao, Sunil Patel, and Stefano Pasquali. 2024. [HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction](#). In *Proceedings of the 5th ACM International Conference on AI in Finance, ICAIF '24*, pages 608–616. Association for Computing Machinery.
- Zirui Song, Bin Yan, Yuhao Liu, Miao Fang, Mingzhe Li, Rui Yan, and Xiuying Chen. 2025. [Injecting domain-specific knowledge into large language models: A comprehensive survey](#). In *Findings of the Association for Computational Linguistics: EMNLP*

- 2025, pages 25297–25311, Suzhou, China. Association for Computational Linguistics.
- Yashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel Ni, Heung-Yeung Shum, and Jian Guo. 2024. [Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph](#). In *ICLR*, volume 2024, pages 3868–3898.
- Yushi Sun, Kai Sun, Yifan Ethan Xu, Xiao Yang, Xin Luna Dong, Nan Tang, and Lei Chen. 2025. [KERAG: Knowledge-enhanced retrieval-augmented generation for advanced question answering](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 6194–6216. Association for Computational Linguistics.
- Alon Talmor and Jonathan Berant. 2018. [The web as a knowledge-base for answering complex questions](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 641–651, New Orleans, Louisiana. Association for Computational Linguistics.
- Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2025. [HydraRAG: Structured cross-source enhanced large language model reasoning](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 14431–14459, Suzhou, China. Association for Computational Linguistics.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. [MuSiQue: Multi-hop questions via single-hop question composition](#). *Transactions of the Association for Computational Linguistics*, 10:539–554.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. [Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10014–10037. Association for Computational Linguistics.
- Ricardo Usbeck, Xi Yan, Aleksandr Perevalov, Longquan Jiang, Julius Schulz, Angelie Kraft, Cedric Möller, Junbo Huang, Jan Reineke, Axel-Cyrille Ngonga Ngomo, et al. 2024. [Qald-10—the 10th challenge on question answering over linked data: Shifting from dbpedia to wikidata as a kg for kgqa](#). *Semantic Web*, 15(6):2193–2207.
- Song Wang, Junhong Lin, Xiaojie Guo, Julian Shun, Jundong Li, and Yada Zhu. 2025. [Reasoning of large language models over knowledge graphs with super-relations](#). In *ICLR*, volume 2025, pages 4208–4226.
- Xinming Wang, Jian Xu, Sheng Lian, Yi Chen, Haiyang Guo, Fei Zhu, Yuanqi Shao, Minsi Ren, Hongzhu Yi, Aslan H. Feng, Hongming Yang, Tailin Wu, Han Hu, Shiming Xiang, Xu-Yao Zhang, and Cheng-Lin Liu. 2026. [The hitchhiker’s guide to autonomous research: A survey of scientific agents](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 1–20.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. [Self-consistency improves chain of thought reasoning in language models](#). In *ICLR*.
- Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. 2024. [Knowledge graph prompting for multi-document question answering](#). In *AAAI*, volume 38, pages 19206–19214.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). In *NeurIPS*, volume 35, pages 24824–24837. Curran Associates, Inc.
- Xiaojun Wu, Cehao Yang, Xueyuan Lin, Chengjin Xu, Xuhui Jiang, Yuanliang Sun, Hui Xiong, Jia Li, and Jian Guo. 2026. [Think-on-graph 3.0: Efficient and adaptive llm reasoning on heterogeneous graphs via multi-agent dual-evolving context retrieval](#). *Preprint*, arXiv:2509.21710.
- Yasheng Xu, Mingda Li, Yongqiang Tang, Peijie Wang, and Wensheng Zhang. 2026. [Towards open-world retrieval-augmented generation on knowledge graph: A multi-agent collaboration framework](#). In *Proceedings of the ACM Web Conference 2026*, WWW ’26, page 2171–2182, New York, NY, USA. Association for Computing Machinery.
- Cehao Yang, Xiaojun Wu, Xueyuan Lin, Chengjin Xu, Xuhui Jiang, Yuanliang Sun, Jia Li, Hui Xiong, and Jian Guo. 2025. [Graphsearch: An agentic deep searching workflow for graph retrieval-augmented generation](#). *Preprint*, arXiv:2509.22009.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. [HotpotQA: A dataset for diverse, explainable multi-hop question answering](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). In *ICLR*.
- Xi Ye and Greg Durrett. 2022. [The unreliability of explanations in few-shot prompting for textual reasoning](#). In *NeurIPS*, volume 35, pages 30378–30392.
- Wen-tau Yih, Matthew Richardson, Chris Meek, Ming-Wei Chang, and Jina Suh. 2016. [The value of semantic parse labeling for knowledge base question](#)

answering. In *ACL*, pages 201–206, Berlin, Germany. Association for Computational Linguistics.

Chuan Yue Yu, Kuo Zhao, Yuhao Li, Heng Chang, Mingjian Feng, Xiangzhe Jiang, Yufei Sun, Jia Li, Yuzhi Zhang, Qingyun Sun, Jianxin Li, and Ziwei Zhang. 2026. [Graphrag-r1: Graph retrieval-augmented generation with process-constrained reinforcement learning](#). In *Proceedings of the ACM Web Conference 2026, WWW 2026, Dubai, United Arab Emirates, originally scheduled for April 13-17, 2026, rescheduled for June 29 - July 3, 2026*, pages 1398–1409. ACM.

Liangliang Zhang, Zhuorui Jiang, Hongliang Chi, Haoyang Chen, Mohammed ElKoumy, Fali Wang, Qiong Wu, Zhengyi Zhou, Shirui Pan, Suhang Wang, and Yao Ma. 2025a. [Diagnosing and addressing pitfalls in kg-rag datasets: Toward more reliable benchmarking](#). In *NeurIPS*, volume 38, Main Conference. Curran Associates, Inc.

Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Hao Chen, Yilin Xiao, Chuang Zhou, Junnan Dong, Yi Chang, and Xiao Huang. 2025b. [A survey of graph retrieval-augmented generation for customized large language models](#). *Preprint*, arXiv:2501.13958.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025c. [Qwen3 embedding: Advancing text embedding and reranking through foundation models](#). *Preprint*, arXiv:2506.05176.

Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yaliang Li, and Wei Hu. 2025. [Knowledge graph-guided retrieval augmented generation](#). In *NAACL*, pages 8912–8924, Albuquerque, New Mexico. Association for Computational Linguistics.

A CoG Algorithm

We provide the pseudo-code of the CoG framework in Algorithm 1. It outlines the iterative workflow, illustrating how the Planning, Exploration, Synthesis, and Reflection modules coordinate to achieve autonomous knowledge exploration in large-scale knowledge bases.

B Evaluation Dataset

In this section, we provide detailed descriptions of the seven multi-hop question answering datasets used in our experiments, covering both KG-based and text-based multi-hop reasoning tasks.

KG-based QA Datasets. These benchmarks primarily require reasoning over structured facts in the knowledge graph and usually expect entity-centric answers grounded in KG nodes or literal attributes.

Algorithm 1 The CoG Algorithm

Input: Question q , KG $\mathcal{G}$, Text Corpus $\mathcal{D}$, Max Turns M
Output: Answer a
Initialize: $\mathcal{N} \leftarrow \emptyset, \mathcal{P} \leftarrow \emptyset, \mathcal{H} \leftarrow \emptyset$
// Phase 1: Initial Planning (§3.2.1)
 $P_0 = (A_0, Q_0, E_0) \leftarrow \text{INITPLAN}(q)$
for $t = 0$ to $M - 1$ **do**
 // Phase 2: Exploration Dual-Source (§3.2.2)
 $K_t \leftarrow \emptyset$
 for each $(q_i^{(t)}, e_i^{(t)}) \in (Q_t, E_t)$ **do**
 $C_i^{(t)} \leftarrow (q, A_t, q_i^{(t)}, e_i^{(t)})$ // Query context
 $F_i^{KG} \leftarrow \text{EXPLOREKG}(C_i^{(t)}, \mathcal{G})$
 // Entity Link $\rightarrow$ Relation Discovery $\rightarrow$ Fact Prune
 $F_i^{\text{text}} \leftarrow \text{EXPLORETEXT}(C_i^{(t)}, \mathcal{D})$
 // Page Selection $\rightarrow$ Skimming $\rightarrow$ Detailed Reading
 $K_t \leftarrow K_t \cup \{F_i^{KG} \cup F_i^{\text{text}}\}$ // Aggregate dual-source evidence
 end for
 // Phase 3: Synthesis (§3.2.3)
 $J_t, M_t \leftarrow \text{SYNTHESIZE}(q, P_t, K_t, \mathcal{N})$
 // Phase 4: Reflection & Re-Planning (§3.2.3)
 if $J_t = \text{SUFFICIENT}$ **then**
 break // Proceed to Answer Generation
 else if $J_t = \text{INSUFFICIENT_USEFUL}$ **then**
 // Pathway 1: Continue Exploration
 $P_{t+1}, \mathcal{N}, \mathcal{P} \leftarrow \text{CONTINUEPLAN}(q, P_t, M_t, \mathcal{N}, \mathcal{P})$
 else
 // Pathway 2: Strategy Adjustment
 $P_{t+1}, \mathcal{P} \leftarrow \text{ADJUSTPLAN}(q, P_t, M_t, \mathcal{H}, \mathcal{N}, \mathcal{P})$
 end if
 $\mathcal{H} \leftarrow \mathcal{H} \cup \{(P_t, M_t)\}$ // Update Interaction History
end for
// Answer Generation (§3.2.4)
 $a \leftarrow \text{GENERATEANSWER}(q, \mathcal{N}, P_t, M_t)$
return a

WebQSP (Yih et al., 2016) is a widely used knowledge base QA benchmark derived from WebQuestions, containing questions paired with SPARQL queries over Freebase (mapped to Wikidata in our setting). It primarily focuses on questions that require 1-2 hop reasoning over structured KG facts to retrieve target entities.

CWQ (Talmor and Berant, 2018) extends the WebQSP by generating questions with higher complexity. It necessitates multi-hop reasoning (up to 4 hops) and challenges models to handle intricate logical constraints, such as composition, conjunctions, and superlatives over the graph structure.

QALD10-en (Usbeck et al., 2024) is the English subset of the 10th Question Answering over Linked Data challenge, specifically adapted for Wikidata. It is characterized by highly complex natural language questions that map to intricate SPARQL queries. The dataset specifically tests a system’s ability to handle advanced structural operations, such as aggregations, filters, and multi-hop relationships.

KGQAGen (Zhang et al., 2025a) is a highly reli-

able, Wikidata-grounded benchmark generated via an LLM-in-the-loop framework with symbolic verification, featuring a wide range of topics and structural complexities. It systematically addresses the inaccurate annotations and ambiguity in existing evaluation datasets, making it a superior standard for evaluating multi-hop reasoning capabilities in KG-based RAG systems.

Text-based QA Datasets. These benchmarks demand semantic understanding and multi-document synthesis, covering span extraction and more free-form answer types such as dates, numbers, boolean answers, and short descriptive phrases.

2WikiMQA (Ho et al., 2020) is a large-scale dataset synthesized from Wikidata and Wikipedia, specifically designed to test multi-hop reasoning across multiple documents. It focuses on questions that require comparison, aggregation, and bridging reasoning types. The questions typically require reasoning chains of 2 to 4 hops, challenging models to perform cross-document integration and logical inference over unstructured text.

MusiQue (Trivedi et al., 2022) is a highly challenging multi-hop QA dataset constructed by compositing single-hop questions in a bottom-up manner. It is explicitly designed to minimize reasoning shortcuts and enforce connected reasoning, ensuring that answering the final question strictly requires resolving intermediate sub-questions. The dataset features 2-4 hop questions that are significantly harder to solve via disconnected reasoning compared to HotpotQA or 2WikiMQA.

AdvHotpotQA (Ye and Durrett, 2022) is an adversarial version of the HotpotQA (Yang et al., 2018) dataset. It introduces distractor paragraphs that share high lexical overlap with the question but do not contain the answer, rigorously testing the system’s robustness against retrieval noise and its ability to identify supporting evidence accurately.

Table 6 summarizes the statistics of the evaluation datasets. For WebQSP, AdvHotpotQA, and QALD10-en, we use the same test splits as in ToG-2 (Ma et al., 2025b) for fair comparison. For CWQ, we adopt the first 1,024 questions from the test set used in ToG (Sun et al., 2024). For 2WikiMQA and MusiQue, we employ the evaluation sets provided by IRCOT (Trivedi et al., 2023). For KGQAGen, we use the full test set from the KGQAGen-10k benchmark.

Category	Dataset	# Questions
KG-based QA	WebQSP	1,430
	CWQ	1,024
	QALD10-en	333
	KGQAGen-10k	1,079
Text-based QA	2WikiMQA	500
	MusiQue	500
	AdvHotpotQA	308

Table 6: Statistics of the evaluation datasets.

C Knowledge Bases

CoG operates over a global knowledge environment that combines structured symbolic facts from Wikidata with entity-centric textual evidence from Wikipedia. This design allows the model to alternate between precise graph-based reasoning and flexible textual exploration, which is important for open-domain multi-hop questions where either source alone may be incomplete.

Structured Knowledge Graph (Wikidata). We use the full Wikidata dump¹ (May 2024) as the structured knowledge graph. After preprocessing, the resulting graph contains 89.3 million entities and 1.38 billion edges. We distinguish two types of edges: relational edges between entities (e.g., (*Steve Jobs*, *founded*, *Apple*)), and attribute edges that connect entities to literal values such as dates, quantities, or strings (e.g., (*Steve Jobs*, *date of birth*, *1955-02-24*)). In total, the processed graph contains 823.7 million relational triples and 562.0 million attribute triples.

This large-scale setting is substantially different from task-specific or locally constructed KGs. It requires entity linking and retrieval to operate over a global entity space rather than a small candidate graph.

Knowledge Graph Preprocessing. We process Wikidata into a retrieval-oriented representation to efficiently support CoG’s core operations.

For *entity grounding*, we extract English labels, aliases, and descriptions. These are encoded via Qwen3-Embedding-4B into a FAISS index, enabling dense candidate recall across nearly 90 million entities with only ~10GB of storage. Since many Wikidata entities share similar or identical

¹<https://dumps.wikimedia.org/wikidatawiki/entities/>

names, each candidate is further supplemented with a compact entity profile, including textual metadata, popularity signals, and local graph context, for fine-grained LLM-based disambiguation.

For *relation exploration and fact acquisition*, we organize relational and attribute triples into an in-memory key-value database (Redis) for efficient neighbor lookups. Additionally, we pre-compute the global occurrence frequencies of all relations, which serves as a crucial structural heuristic for the agent to prune high fan-out paths during exploration.

Unstructured Text Corpus (Wikipedia). We employ the real-time English Wikipedia² as our unstructured text corpus. It comprises approximately 7.1 million entity-centric articles, totaling over 5 billion words. Distinct from traditional RAG that chunks text flatly, we process each page to retain its hierarchical structure, including the summary paragraph, table of contents, and sectioned full text. This structure supports CoG’s hierarchical reading strategy (global skimming followed by detailed reading), enabling efficient navigation through long documents.

Wikipedia complements Wikidata by providing richer natural-language context, especially for facts that are difficult to normalize into triples. In contrast, Wikidata provides normalized entities and typed relations for precise symbolic traversal. CoG exploits this complementarity by using KG facts as high-precision reasoning anchors and Wikipedia passages as high-recall contextual evidence.

D Additional Evaluation Metrics

To complement the primary EM evaluation, Tables 7 and 8 report LLM-as-judge CORRECT rates and token-level F1 scores, respectively, for the same model outputs as in Table 2. For LLM-as-judge, Qwen3.5-122B-A10B (non-thinking mode, temperature 0) classifies each response as CORRECT, PARTIAL, INCORRECT, or NO ANSWER, given only question, reference answers, and model response. We conservatively report only the CORRECT rate. For token-level F1, the same model extracts a concise final answer using only the question and response. We then normalize predictions and references by lowercasing, removing punctuation and articles, and normalizing whitespace. For multiple references, we retain the maximum F1.

²<https://en.wikipedia.org/w/api.php>

As shown in Tables 7 and 8, the complementary metrics are consistent with the EM-based findings. Under LLM-as-judge, CoG achieves the best result on all seven datasets, with an average CORRECT rate of 61.63%, outperforming the strongest baseline, IRCOT, by 12.10 points. Under token-level F1, CoG also obtains the highest average score of 49.87%, improving over ToG-2 by 4.49 points. These results show that CoG’s gains are not merely caused by exact string matching, but remain clear under both semantic equivalence judgment and lexical-overlap evaluation.

E Detailed Results on Backbone LLMs

This section supplements the backbone LLM generalizability analysis in Section 4.4. Table 9 presents the detailed performance breakdown across different backbone LLMs, corresponding to the summarized results in Table 4.

Evaluation Setup. To balance computational cost with evaluation rigor, we adopted the following dataset selection and sampling strategies:

Dataset Selection. We excluded the WebQSP dataset from this analysis, as modern LLMs already achieve near-saturated performance on its simple 1-2 hop queries in the Direct Prompting setting, making it less discriminative for measuring improvement.

Open-source Models. For large-scale KG-based datasets (KGQAGen and CWQ), we sampled 300 instances each to align with the scale of QALD10-en. For the remaining datasets (QALD10-en, 2WikiMQA, AdvHotpotQA, and MusiQue), we evaluated on the full test sets.

Proprietary Models. We evaluated on a sampled subset of 100 examples for each dataset.

Key Observations. The detailed results reveal three key insights regarding CoG’s generalizability and cost-effectiveness: (1) **Broad Generalizability:** CoG consistently improves performance across all backbone models, regardless of their scale (from 8B to 235B) or type (open-source vs. proprietary). (2) **Unlocking Small Models:** The relative performance gain is most pronounced on the smallest model, Qwen3-8B (+113.2%), showing that CoG’s cognitive architecture effectively compensates for the limited parametric knowledge of smaller LLMs. (3) **Cost-Effectiveness:** CoG-8B (Avg. 51.0%) outperforms direct prompting on massive models (e.g. Qwen3-235B 43.7%, DeepSeek-V3.2

Baseline Type	Method	Multi-hop KG-based QA				Multi-hop text-based QA			Avg.
		KGQAGen	CWQ	QALD10-en	WebQSP	2WikiMQA	AdvHotpotQA	MusiQue	
LLM-only	Direct	38.74	34.57	40.84	64.06	29.20	23.38	14.20	35.00
	CoT	38.28	35.06	44.14	60.28	31.20	25.97	15.40	35.76
	CoT-SC	39.48	32.71	43.54	61.05	32.40	26.62	14.80	35.80
Text-based RAG	Vector RAG	50.23	33.20	37.84	64.83	48.80	45.45	18.20	42.65
	ReAct	48.19	35.74	42.04	69.79	57.60	54.87	22.80	47.29
	IRCoT	52.55	34.57	37.84	66.43	68.60	61.69	25.00	49.53
	Search-o1	52.27	39.84	48.65	68.60	47.80	47.73	27.40	47.47
KG-based RAG	ToG	42.63	29.20	45.95	62.87	34.80	22.08	11.20	35.53
	PoG	51.44	29.59	49.85	70.42	79.20	19.80	16.40	45.24
	DoG	43.65	25.68	51.65	63.29	62.00	24.68	13.80	40.68
Hybrid RAG	ToG-2	53.57	41.50	49.85	70.21	65.20	42.86	22.40	49.37
Proposed	CoG	66.82	44.92	59.76	76.57	88.20	62.34	32.80	61.63

Table 7: LLM-as-judge CORRECT rate (%) on seven datasets. The best result in each column is highlighted in **bold**.

Baseline Type	Method	Multi-hop KG-based QA				Multi-hop text-based QA			Avg.
		KGQAGen	CWQ	QALD10-en	WebQSP	2WikiMQA	AdvHotpotQA	MusiQue	
LLM-only	Direct	38.64	33.17	28.14	49.59	33.19	21.63	15.77	31.45
	CoT	40.28	35.25	34.35	55.96	35.94	25.13	17.68	34.94
	CoT-SC	41.91	34.59	35.29	56.30	36.14	24.58	18.11	35.27
Text-based RAG	Vector RAG	48.49	28.98	24.43	47.81	44.61	36.84	17.50	35.52
	ReAct	46.15	29.90	25.58	49.78	51.55	42.78	21.99	38.25
	IRCoT	51.26	30.34	25.19	48.08	59.86	47.93	23.13	40.83
	Search-o1	49.18	35.43	31.39	51.48	44.22	39.44	25.65	39.54
KG-based RAG	ToG	43.31	27.24	35.58	58.21	34.31	19.51	13.22	33.05
	PoG	51.30	26.46	39.84	59.35	73.47	16.47	15.56	40.35
	DoG	46.00	27.20	43.20	56.01	61.84	21.15	14.35	38.54
Hybrid RAG	ToG-2	54.96	39.46	38.55	60.58	62.80	37.85	23.45	45.38
Proposed	CoG	67.69	38.46	37.96	50.36	76.22	49.95	28.48	49.87

Table 8: Token-level F1 (%) on seven datasets. The best result in each column is highlighted in **bold**.

38.0%) and surpasses strong 32B baselines (e.g., IRCoT 44.6%, ToG-2 45.9%). Since 8B models are cheaper and faster, CoG-8B offers a highly cost-effective deployment solution that neutralizes the token overhead of multi-turn exploration.

F Detailed Results on Exploration Depth

Table 10 provides the full results corresponding to the exploration depth analysis discussed in Section 4.4 and visualized in Figure 3. The table lists the Exact Match (EM) scores for CoG across exploration depths from 1 to 6 on all seven benchmark datasets, alongside the average performance on KG-based and Text-based tasks.

G Detailed Efficiency Profile

This section details CoG’s computational overhead across token consumption, end-to-end latency, and wall-clock time per retrieval round.

Complexity Adaptability. CoG dynamically scales resources to query difficulty (Table 11). For simpler tasks (WebQSP, 2WikiMQA), it uses fewer

tokens (~ 14 - 15 k) than IRCoT (~ 22 k) yet achieves higher accuracy. For complex tasks (MusiQue, KGQAGen), it invests more tokens into deep reasoning, yielding substantial gains where efficient methods fail (+11.6% on MusiQue over ToG-2).

Latency Breakdown. Figure 4 details the average wall-clock time per retrieval round. The latency primarily stems from sequential LLM calls in the cognitive cycle. However, this extended reasoning time supports a fully open-ended retrieval setting: CoG starts from the raw question without pre-annotated entities, directly grounds and retrieves evidence, and adaptively adjusts KG/text exploration according to the evolving reasoning state. Furthermore, this deep filtering ensures precise localization, reducing accessed documents by $12.4\times$ compared to ToG-2 (Table 5). The current wall-clock time can be further optimized by parallelizing the dual-source exploration modules and concurrent sub-queries.

Cost-Effectiveness. While CoG consumes more tokens on complex tasks, this is offset by its ability

Backbone	Method	KG-based QA			Text-based QA			Average		
		KGQAGen	CWQ	QALD	2Wiki	Hotpot	MusiQ	KG	Text	Total
Open-Source Models										
Qwen3-32B	Direct	40.00	37.00	47.45	47.20	20.13	8.40	41.48	25.24	33.36
	IRCoT	53.80	33.30	41.10	71.60	47.10	20.60	42.73	46.43	44.58
	Search-o1	46.34	36.72	43.54	44.60	40.26	19.40	42.20	34.75	38.48
	DoG	54.80	33.10	55.90	61.20	20.10	11.40	47.93	30.90	39.42
	ToG-2	55.60	41.40	53.50	73.00	35.70	16.20	50.17	41.63	45.90
	CoG	76.33	46.33	55.26	85.20	51.95	27.80	59.31	54.98	57.15
Qwen3-8B	Direct	29.10	27.83	36.34	31.80	12.99	5.40	31.09	16.73	23.91
	CoG	68.03	42.29	49.85	80.00	45.45	20.20	53.39	48.55	50.97
Qwen3-235B	Direct	54.67	44.33	54.95	55.20	32.47	20.80	51.32	36.16	43.74
	CoG	79.00	55.11	56.46	87.60	54.55	32.80	63.52	58.32	60.92
gpt-oss-120b	Direct	18.67	29.67	36.64	20.80	18.83	8.00	28.33	15.88	22.10
	CoG	42.33	40.33	45.95	63.40	38.64	18.80	42.87	40.28	41.58
DeepSeek-V3.2	Direct	53.67	45.33	44.74	39.20	31.17	13.80	47.91	28.06	37.99
	CoG	85.33	51.67	57.36	88.40	59.09	34.00	64.79	60.50	62.64
Proprietary Models (Sampled Set, N=100)										
GPT-5.1	Direct	54.00	47.00	36.00	59.00	40.00	17.00	45.67	38.67	42.17
	CoG	81.00	49.00	38.00	82.00	55.00	29.00	56.00	55.33	55.67
Gemini-2.5-Flash	Direct	62.00	46.00	27.00	65.00	46.00	22.00	45.00	44.33	44.67
	CoG	73.00	44.00	31.00	78.00	58.00	23.00	49.33	53.00	51.17

Table 9: Detailed performance comparison (EM, %) of different backbone LLMs with and without the CoG framework across six datasets (WebQSP excluded). For Qwen3-32B, we additionally include strong baselines from various categories (Text-based, KG-based, and Hybrid RAG) to demonstrate that CoG with a lightweight 8B model can outperform these 32B baselines.

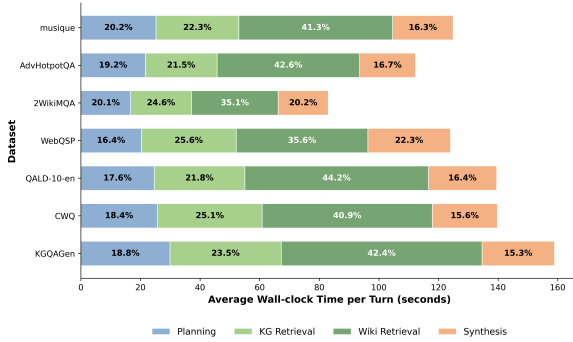


Figure 4: Average wall-clock time (seconds) breakdown per retrieval round across different datasets.

to utilize smaller, cheaper models. As shown in Appendix E, CoG with an 8B model outperforms strong 32B baselines, suggesting a more favorable accuracy-cost trade-off for practical deployment.

H Sensitivity Analysis on Entity Linking

To evaluate the robustness of CoG to the candidate retrieval size (top- k) in the entity linking module, we vary the number of candidate entities from 6 to 30. We conduct this analysis on four representative

datasets, covering both KG-based and text-based QA tasks. For KGQAGen, we evaluate on a 300-sample subset to control computational cost.

As shown in Table 12, CoG is robust to the entity-linking candidate size. Across $k = 6$ to 30, the average EM remains within a narrow range of 59.1%–60.3%. Even with a narrow retrieval scope ($k = 6$), CoG achieves a strong average EM of 59.4%, while increasing k provides broader entity recall without causing overall performance degradation. We set $k = 20$ as the default because it achieves the best average performance, tied with $k = 30$, while using fewer candidate entities. This indicates that CoG’s context-aware disambiguation effectively filters noisy candidates while benefiting from broader entity recall.

I Manual Evaluation and Error Study

As the Exact Match (EM) metric strictly relies on string matching, it often fails to evaluate the validity of the underlying reasoning process. To gain deeper insights, we conduct a manual analysis on a sampled subset. We assess **Reasoning Faithfulness** for cases where the EM score is correct (EM=1),

Method	KG-based QA					Text-based QA				
	KGQAGen	CWQ	QALD10	WebQSP	Avg.	2Wiki	AdvHotpot	MusiQue	Avg.	Avg.
VectorRAG	48.66	30.76	39.04	70.49	47.24	52.60	32.14	12.40	32.38	40.87
CoG (Depth=1)	67.47	37.99	45.35	80.63	57.86	48.00	26.62	4.60	26.41	44.38
CoG (Depth=2)	71.83	42.58	51.95	81.05	61.85	85.20	46.43	20.00	50.54	57.01
CoG (Depth=3)	74.24	42.58	53.75	81.75	63.08	86.80	49.35	25.00	53.72	59.07
CoG (Depth=4)	74.70	42.87	55.26	81.75	63.65	85.20	51.95	27.80	54.98	59.93
CoG (Depth=5)	74.51	42.97	51.65	81.47	62.65	84.00	52.92	26.00	54.31	59.07
CoG (Depth=6)	75.90	43.65	54.35	80.56	63.62	83.20	53.25	25.60	54.02	59.50

Table 10: Full results of exploration depth analysis. We report Exact Match (EM) scores for CoG with maximum exploration depths ranging from 1 to 6 on all seven datasets, alongside the VectorRAG baseline. The best result for each dataset is highlighted in **bold**.

Method	KGQAGen		CWQ		QALD10-en		WebQSP		2WikiMQA		AdvHotpotQA		MusiQue	
	Token	Latency	Token	Latency	Token	Latency	Token	Latency	Token	Latency	Token	Latency	Token	Latency
ReAct	3,850	14.1	3,996	13.7	3,908	13.8	3,870	12.3	3,984	18.8	3,631	13.2	7,758	20.0
IRCoT	23,832	32.9	26,107	28.9	23,483	33.7	22,798	19.5	22,350	41.5	23,038	38.2	29,798	46.1
Search-o1	12,262	98.4	13,505	119.8	10,611	84.0	9,891	62.6	19,196	140.5	11,932	77.9	20,913	120.0
ToG	6,552	68.7	4,168	41.9	4,154	69.7	2,963	39.1	4,779	70.5	4,946	68.4	4,695	75.2
PoG	8,752	16.7	5,117	12.7	3,660	10.9	2,862	7.0	4,909	16.6	7,220	23.7	11,899	28.4
DoG	15,494	168.7	10,304	133.9	3,088	20.6	3,320	18.1	6,671	105.4	10,364	158.7	8,575	134.0
ToG-2	4,193	109.7	3,424	78.6	3,273	53.3	3,127	51.3	3,067	42.7	3,398	51.0	4,008	95.0
CoG	36,323	285.1	29,465	194.2	15,636	95.7	14,204	80.7	15,508	121.8	18,958	145.8	38,876	263.5

Table 11: Token consumption and latency (seconds) per question across different methods. Values represent the median of the dataset to reflect the typical cost for a standard query.

k	KGQAGen	2Wiki	Hotpot	MusiQ	Avg.
6	79.3	84.8	47.7	25.6	59.4
12	78.3	84.2	49.7	25.2	59.4
18	74.3	85.8	50.0	26.2	59.1
20	76.3	85.2	52.0	27.8	60.3
30	78.0	85.6	51.3	26.2	60.3

Table 12: Sensitivity analysis of entity-linking candidate size k . We report EM (%) on four representative datasets.

and perform an **Error Diagnosis** for cases where the model is judged incorrect (EM=0).

1.1 Reasoning Faithfulness

For predictions judged correct by EM, we further examine whether the model reaches the answer through a faithful reasoning process. We categorize each EM-correct prediction into four types:

- **True Positive (TP):** The reasoning chain is logically sound, and the extracted evidence fully supports the final answer.
- **Flawed True Positive (Flawed TP):** The final answer and core reasoning direction are correct, but the reasoning contains minor verification gaps, shortcut steps, or non-critical factual imperfections.

- **Weak False Positive (Weak FP):** The final answer is correct by coincidence, but the core intermediate entities or facts are substantially incorrect, i.e., the model is right for the wrong reasons.

- **False Positive (FP):** The model fails to deduce the correct answer, but the EM metric falsely judges it correct due to substring matching in the reasoning text.

Table 13 shows that CoG produces the most faithful EM-correct predictions. Its severe false-positive rate (Weak FP + FP) is only 3.6%, much lower than ToG-2 (12.7%) and Self-Consistency (29.1%). Even when Flawed TP is treated as non-ideal reasoning, CoG remains the most reliable method.

Self-Consistency is vulnerable to ungrounded parametric reasoning, often producing hallucinated bridge entities or fabricated attributes (*Weak FPs*), or long CoT traces that accidentally match the gold string (*FPs*). ToG-2 reduces such hallucinations through retrieval, but its reactive exploration can still preserve wrong intermediate entities or unresolved constraints. CoG’s graph-text exploration and reflection better ground intermediate steps, so most of its imperfect cases involve incomplete verification of secondary constraints (*Flawed TPs*) rather than broken core reasoning chains.

Method	TP	Flawed TP	Weak FP	FP
Self-Consistency	60.0%	10.9%	21.8%	7.3%
ToG-2	81.8%	5.5%	3.6%	9.1%
CoG (Ours)	89.1%	7.3%	1.8%	1.8%

Table 13: Distribution of reasoning faithfulness for predictions evaluated as correct by EM.

Question: What nationality is the sport club that the current coach of Werder Bremen played professionally for? Gold Answer: German Reference Reasoning Path: Werder Bremen → (coach) Alexander Nouri → (played for) VfL Osnabrück → (nationality) German ✗ SC (Weak FP): Werder Bremen → Florian Kohfeldt → [Hallucinated] VfL Wolfsburg → German Hallucinates that coach F. Kohfeldt played for VfL Wolfsburg (he only coached them). Hits the answer “German” purely by coincidence. ✗ ToG-2 (Weak FP): Werder Bremen → Florian Kohfeldt → [Guessed] Werder Bremen → German Fails to retrieve his actual former clubs. Circularly assumes that since he coaches Werder Bremen (a German club), he must have played for them, guessing “German”. ✓ CoG (True Positive): Werder Bremen → Daniel Thioune → VfL Osnabrück → German Identifies current coach D. Thioune from updated knowledge. Retrieves his actual former clubs (e.g., VfL Osnabrück) and confirms they are German.
--

Figure 5: A qualitative example showing CoG’s superior reasoning faithfulness despite identical EM scores across models.

Figure 5 further shows that identical EM scores can hide different reasoning quality. While SC and ToG-2 may reach the correct answer through hallucinated or weakly supported intermediate links, CoG derives the answer from a more grounded and faithful reasoning path. This example also involves a temporal update: although the dataset reference path is based on an older coach, CoG identifies the updated coach and reaches the correct nationality through verifiable evidence.

I.2 Error Diagnosis

To understand the limitations of CoG, we conduct a primary-cause error diagnosis on a sampled subset of instances where CoG fails the EM evaluation. As illustrated in Figure 6, we categorize the errors into External Factors and Internal Errors.

External Factors (71.1%): A large portion of the errors are attributable to benchmark or evaluation limitations.

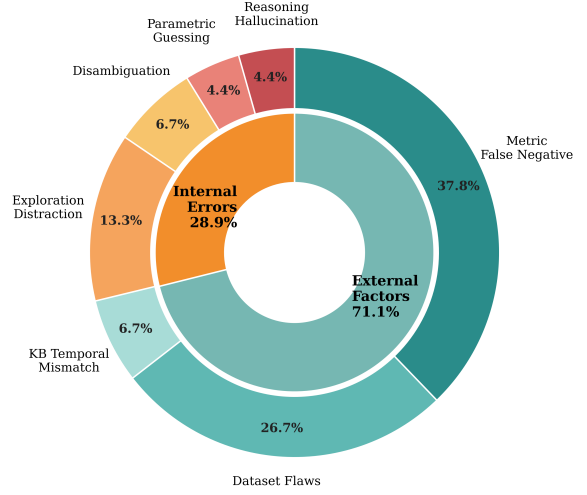


Figure 6: Taxonomy of error causes for CoG based on manual analysis.

- **Metric False Negative (37.8%):** The model successfully completes the multi-hop reasoning but fails the strict string matching. Common instances include surface-form mismatch (aliases or pseudonyms), paraphrases, or answer granularity differences (city vs. metro area).
- **Dataset Flaws (26.7%):** These include typos in the questions, grammatically ambiguous prompts leading to misaligned relation extraction, and noisy ground-truth annotations.
- **KB Temporal Mismatch (6.7%):** Temporal mismatch occurs when the model retrieves updated facts from the knowledge source (e.g., recent sports team transfers), which conflict with the outdated static dataset ground truth.

Internal Errors (28.9%): The remaining errors represent genuine limitations in the agentic exploration process.

- **Exploration Distraction (13.3%):** In environments with long texts or ambiguous clues, the agent occasionally gets trapped by irrelevant entities or over-thinks historical details, exhausting its maximum exploration turns.
- **Disambiguation (6.7%):** The model sometimes overlooks rigid constraints during entity

disambiguation (e.g., ignoring a "state school" constraint and erroneously selecting a private school).

- **Reasoning Hallucination (4.4%) & Parametric Guess (4.4%):** The model occasionally struggles with counting logic over complex long tables, or bypasses retrieval entirely by relying on over-confident parametric guesses.

Overall, this diagnosis shows that raw EM can underestimate CoG, with many failures caused by rigid string matching, noisy annotations, or temporal mismatch. The remaining internal errors highlight important directions for improvement in exploration focus, entity disambiguation, and evidence-grounded verification.

J Case Studies

Table 14 and Table 15 compare the reasoning trajectories of ToG-2 and CoG on a complex multi-hop question from AdvHotpotQA.

ToG-2 Analysis. It fails due to its passive, graph-driven exploration strategy. The system starts by retrieving general information about “Stanford Cardinal” but fails to locate the specific player due to the absence of statistical attributes in the KG. Lacking a reflection mechanism, ToG-2 cannot diagnose this failure. Instead, it blindly continues its beam search, pivoting to high-ranking but irrelevant entities like “quarterback” in Turn 3, ultimately leading to an irreversible dead end.

CoG Analysis. In contrast, CoG demonstrates robust navigation through its cognitive cycle and bidirectional synergy. CoG initially encounters the same dead end as ToG-2 in Turn 1. However, Reflection diagnoses the strategy as too narrow. Proactive Planning then pivots to broader queries like “football receiving leaders” instead of the player directly (Turn 2). Crucially, Text Entity Utilization bridges KG gaps by identifying the entity “James Lofton”, which aligns with the statistical constraints in the question, from Wikipedia related sections. This bidirectional synergy enables CoG to dynamically identify the target player and verify the specific statistics in Turn 3, successfully deriving the answer “David”.

K Prompts

This section provides the main prompt used in CoG to guide LLM reasoning at each stage of the cognitive cycle.

Table 16 presents the prompt used for the **initial planning phase** (Section 3.2.1). The prompt includes four examples to guide the LLM in handling diverse multi-hop reasoning scenarios.

Tables 17–20 present the prompts for the three-stage **KG Exploration** (Section 3.2.2): Entity Linking, Relation Discovery, and Fact Pruning, respectively. These prompts demonstrate how CoG leverages in-context learning to achieve precise, schema-aware navigation over large-scale KGs.

Tables 21–24 detail the prompts used in the **Text Exploration module** (Section 3.2.2), covering Adaptive Page Selection (Tables 21 and 22) and Hierarchical Reading (Tables 23 and 24).

Table 25 presents the prompt for the **Synthesis module**. Tables 26 and 27 provide examples of structured evidence formats for KG and text, respectively, that populate the `{evidence_blocks}` placeholder in the Synthesis prompt.

Tables 28 and 29 detail the prompts for the **Reflection module** (Section 3.2.3), enabling the system to either deepen exploration based on new findings or pivot its strategy by diagnosing failures from the interaction history.

Question: *What is the middle name of the former Stanford Cardinals player who caught 57 passes for 1010 yards and 14 TDs?*

Pre-annotated Entities: Stanford Cardinal (Q2938060), Touchdown (Q650807)

Ground Truth: David

Turn	ToG-2 Reasoning Process
turn 1	Starting Entities: Stanford Cardinal (Q2938060), Touchdown (Q650807) Relations Explored: member of sports team, sport, educated at, facet of, named after (6 relations) Retrieved Entities: 57 entities (49 from “member of sports team”, 4 from “sport”, 1 from “educated at”, 2 from “Touchdown” relations) Notable: 49 Stanford athletes including Greg Camarillo, John Elway, Mark Marquess, etc. Document Retrieval: Retrieved Wikipedia pages for 55 entities (2 pages failed: sports terminology, Margaret Hayden) Issue: No specific statistics matching 57/1010/14 found in documents Reasoning: Information insufficient - cannot identify the specific player Entity Ranking (for next turn): [0.28] Greg Camarillo [0.12] John Elway [0.05] Mark Marquess
turn 2	Starting Entities: Greg Camarillo (Q1248387), John Elway (Q432931), Mark Marquess (Q6768732) Relations Explored: given name, educated at, member of sports team, position played, occupation (9 relations) Retrieved Entities: 11 entities (Greg, Menlo-Atherton HS, Stanford Univ., 5 NFL teams, John, quarterback, Mark, baseball player, Stanford Cardinal) Document Retrieval: Retrieved Wikipedia pages for all 11 entities Issue: Found Greg Camarillo’s given name but no middle name; no confirmation of statistics Reasoning: Cannot confirm Greg Camarillo is the target player or find his middle name Entity Ranking (for next turn): [0.45] Stanford Cardinal [0.00] quarterback
turn 3	Starting Entities: Stanford Cardinal (Q2938060), quarterback (Q622747) Relations Explored: member of sports team, position played, sport (6 relations) Retrieved Entities: 106 entities (50 from “member of sports team”, 4 from “sport”, 1 from “educated at”, 50 from “position played”, 1 from “part of”) Notable: 50 quarterbacks (Logan Thomas, John Brantley, Charlie Whitehurst, etc.), 50 Stanford athletes Document Retrieval: Retrieved Wikipedia pages for 103 entities (3 pages failed: Cameron Dantley, Dai Minagawa, Margaret Hayden) Critical Failure: Explored wrong position (quarterback vs. wide receiver) No Reflection: System continued beam search without recognizing strategic misalignment Entity Ranking: [0.31] Logan Thomas [0.12] John Brantley [0.02] Charlie Whitehurst Final Reasoning: Information insufficient to identify the player or middle name Outcome: FAILURE - Generated answer without evidence

Table 14: Reasoning trace of ToG-2. The method employs a rigid beam search where the next exploration steps are determined by ranking score based on entity document relevance. Without reflection, the pruning mechanism blindly discards potentially correct paths and misguides the search towards irrelevant high-ranking entities, leading to an irreversible failure. Red highlights indicate critical failure analysis for the system.

Question: <i>What is the middle name of the former Stanford Cardinals player who caught 57 passes for 1010 yards and 14 TDs?</i> Ground Truth: David	
Turn	CoG Reasoning Process
Turn 1	Planning Phase: Analysis: “To determine the middle name, I first need to identify the specific player who caught 57 passes for 1010 yards and 14 touchdowns.” Query: “Stanford Cardinals player with 57 catches, 1010 yards, 14 TDs” Entity: “Stanford Cardinals player” Exploration Phase: KG: Entity linking failed - 9 candidates all team-level entities (e.g., Stanford Cardinal football, Stanford Cardinal men’s soccer), no individual player entities Wikipedia: Searched “Stanford Cardinal football” page, extracted team context (Jim Plunkett, Andrew Luck) but no player matching the statistics Synthesis: No player-specific information found Reflection: Judgment = INSUFFICIENT_USELESS “The initial approach was too narrow... The core strategic error was not breaking the problem into smaller, more general steps.”
Turn 2	Recovery Planning: Strategy Shift Reflection: “Instead of starting with the player’s name, we should first identify the season or year when these stats might have been achieved, or search for a list of Stanford receivers with notable seasons.” Updated Analysis: “Shift focus to general sources of Stanford football statistics or notable players.” Next Queries: [“Stanford football all-time receiving leaders”, “Stanford football players with 14 TDs in a season”] Exploration (Query: “Stanford football all-time receiving leaders”): KG: Linked to “Stanford Cardinal football statistical leaders” (Q19878473) Relation Discovery: Found 1 relation - “sport” (P641) → American football Fact Pruning: Kept 0 facts (too general, not useful for identifying specific players) Wikipedia: Found “Stanford Cardinal football statistical leaders” page with “Receiving” section “Receiving” section containing 9 statistical tables, extracted the following information from table: “Single season 6” (receiving yards): James Lofton - 1010 yards (1977) “Single season 7” (receiving TDs): James Lofton - 14 TDs (1977); J.J. Arcega-Whiteside - 14 TDs (2018) Key Discovery: Identified “James Lofton” as candidate with 2/3 statistics matched (1010 yards + 14 TDs in 1977) Synthesis: Strong candidate found but 57 receptions unconfirmed Reflection: Judgment = INSUFFICIENT_USEFUL - Continue with James Lofton
Turn 3	Planning Phase: Query: “James Lofton 1977 season stats” Entities: “James Lofton” Exploration Phase: KG: Linked to James Lofton (Q1680714) Identity Check: educated at → Stanford; member of team → Stanford Cardinal. Attribute Retrieval: Found alias “James David Lofton” (Directly reveals middle name) Wikipedia: Found “James Lofton” page From “College career” section: Confirmed “57 receptions for 1, 010 yards and 14 touchdowns during the 1977 season” Synthesis: KG confirmed identity & name; Text verified the missing statistical constraint. Middle name derivation, complete. Reflection: Judgment = SUFFICIENT Final Answer: “The middle name of the former Stanford Cardinals player is David.” Outcome: SUCCESS in 3 turns

Table 15: Reasoning trace of CoG. Reflection and Proactive Planning diagnose the initial failure and recover from dead ends. Adaptive Entity Linking identifies implicitly useful entities (“1977 Stanford Cardinals football team”) not directly mentioned in the question. Finally, Text Entity Utilization effectively extracts the target entity (“James Lofton”) from text to guide the final answer derivation. Purple highlights show successful reflection and strategic adjustment; blue highlights show key information extraction from dual sources (KG structure + Wikipedia text).

ROLE

You are a highly strategic retrieval planner. Your function is to analyze a complex question and identify the immediate, parallelizable search actions required to proceed.

TASK

Given a user's question, do NOT provide the final answer. Instead, determine the next logical search queries. A single question might require multiple parallel queries to gather the necessary initial information. Your plan must include your reasoning, a list of search queries, and a corresponding list of entities.

CONSTRAINTS

- The number of queries in the 'Query' list must not exceed 5.
- **[CRITICAL]** The 'Query' and 'Entities' lists must have the exact same number of items. For each query, the corresponding item in the 'Entities' list must be the **single, most specific, and central named entity** being investigated in that query.
- Only output the query/queries for the immediate next step. Do not plan, describe, or allude to any subsequent steps.

OUTPUT FORMAT

Your output must follow this exact structure, with no additional commentary:

Analysis: Your thought process explaining why these next steps are necessary and what information is being sought.

Query: [A Python-style list of concise and effective search query strings.]

Entities: [A Python-style list of the core entities in the Query, corresponding one-to-one with the Query list.]

EXAMPLES

Example 1:

Question: What is the nationality of one of the men for whom the Schumann–Runge bands are named?

Output:

Analysis: The question requires the nationality of a person. Before I can find their nationality, I must first identify the individuals the "Schumann–Runge bands" are named after.

Query: ["who are the Schumann–Runge bands named after"]

Entities: ["Schumann–Runge bands"]

... (Other examples omitted for brevity) ...

Example 4:

Question: Where did the leader of the largest European country after the collapse of the country that denied anything more than an advisory role in the Korean war die?

Output:

Analysis: The question is complex and requires multiple pieces of information. First, I need to identify which country denied anything more than an advisory role in the Korean War. [...] I will begin by identifying the country that denied a combat role in the Korean War.

Query: ["country that denied anything more than an advisory role in Korean War"]

Entities: ["Korean War"]

YOUR TASK

Question: \${question}

Output:

Table 16: Prompt template for the Initial Planning module. The LLM is instructed to analyze the question, generate parallelizable sub-queries, and identify corresponding anchor entities. This structured output initializes the exploration cycle.

ROLE

You are an expert in Knowledge Graph entity linking. Your task is to disambiguate an entity mention from a user's question by matching it to the correct entity in a knowledge graph, using the surrounding context.

CONTEXT

- **Original Question:** \${question}
- **Overall Plan (Analysis):** \${analysis}
- **Current Sub-Query:** \${query}
- **Entity Mention to Link:** \${entity}

CANDIDATE ENTITIES

Here are the top candidate entities in the Knowledge Graph, sorted by a preliminary relevance score. Each candidate includes its Wikidata QID, label, description, aliases, popularity, some of its neighbors, and relevance score.

`${formatted_candidates}`

YOUR TASK

Your task is to critically evaluate the candidate entities based on the provided CONTEXT. Your goal is to either identify the single correct entity OR determine that no suitable match exists.

- **Analyze the Context:** Carefully review the 'Original Question' and 'Overall Plan'. What are the key details about the entity \${entity} (e.g., their time period, relationships, role)?
- **Evaluate Each Candidate:** For each candidate, compare its Description, Aliases, and Neighborhood against the context. A correct match should be consistent with the context.
- **Make a Decision:**
 - **If, and only if,** you find one candidate that is a confident and accurate match for \${entity} based on all available information, your output should be its QID on a single line.
 - **If none of the candidates are a confident match,** or if their key details contradict the context, you **MUST** output the single word: NO_MATCH.

OUTPUT FORMAT

- If a confident match is found, your entire output **MUST** be only the QID of that entity (e.g., Q12345).
- If no confident match is found, your entire output **MUST** be the single word: NO_MATCH.
- Your output must not contain any other text, explanation, or reasoning.

YOUR RESPONSE:

Table 17: Prompt template for the Entity Linking module. The LLM is tasked with precise disambiguation by comparing candidate profiles against the question context. The placeholder `${formatted_candidates}` is populated with structured candidate data, as illustrated in Table 18.

Example of Formatted Candidates

```
-
[Entity]
- QID: Q716680
- Label: Steve McQueen
- Description: American actor (1930-1980)
- Aliases: Terrence Stephen McQueen, King of Cool
- Popularity (Degree): In: 154, Out: 45, Attr: 22
[Neighborhood]
- Outgoing Relations (as Head):
- spouse: Neile Adams, Ali MacGraw, Barbara Minty
- child: Chad McQueen, Terry Leslie McQueen
[Scores]
- Final: 0.952 (Name similarity=1.000, Popularity=0.850, Description=0.920, Neighbor=0.880)
-
... (Other candidates omitted) ...
```

Table 18: An example of the formatted candidate data populated into the `${formatted_candidates}` slot in Table 17. This structured format provides the LLM with comprehensive entity details (profile, neighborhood, and retrieval scores) for disambiguation.

ROLE

You are an expert Knowledge Graph analyzer. Your task is to select the most relevant relations (properties) of a given entity that will help answer a specific question.

CONTEXT

- **Original Question:** \${question}
- **Overall Plan (Analysis):** \${analysis}
- **Current Sub-Query:** \${query}
- **Entity in Focus:** \${entity_label} (\${entity_qid}): \${entity_description}

AVAILABLE RELATIONS

The entity \${entity_label} is connected to \${total_relations_count} relations (Outgoing: \${outgoing_count}, Incoming: \${incoming_count}) in the Knowledge Graph. Below is a list of all relations connected to the entity. Each relation includes its label, PID, and a frequency indicating how many other entities in the graph are linked via this relation.

Outgoing Relations:

\${outgoing_relations}

Incoming Relations:

\${incoming_relations}

YOUR TASK

Based on the **CONTEXT**, select relations that will help find the answer using a two-pronged approach:

1. **Direct Relations:** Select relations that seem to directly contain the answer.
2. **Exploratory Relations:** If no direct relations exist, select relations that could lead to intermediate entities, which might then contain the answer.

OUTPUT FORMAT

Your output must follow this exact structure, with no additional commentary:

Reasoning: Your brief thought process explaining why you chose these specific relations based on the question and the entity.

Selected PIDs: [A Python-style list of strings, where each string is the PID of a selected relation (e.g., ["P31", "P17"]). If no relations seem relevant, output []]

YOUR RESPONSE:

Table 19: Prompt template for the Relation Discovery module. The LLM selects a subset of relations relevant to the sub-query to prune the search space at the schema level.

ROLE

You are an expert assistant specializing in knowledge graph analysis for question answering. Your mission is to intelligently prune a list of facts, guiding a multi-step exploration process.

CONTEXT

- **Original Question:** \${question}
- **Overall Plan (Analysis):** \${analysis}
- **Current Sub-Query:** \${query}
- **Entity in Focus:** \${entity_label}

AVAILABLE FACTS

Below is a structured list of all facts retrieved from the Knowledge Graph for the entity \${entity_label}. The facts are categorized by relation type (Outgoing or Incoming).

\${formatted_facts}

YOUR TASK

Your goal is to decide which facts to keep. Evaluate the facts using a two-pronged approach:

1. **Directly Relevant Facts:** Keep any facts that directly help answer the **Current Sub-Query**.
2. **Promising Intermediate Entities:** Keep facts that represent entities which are promising stepping stones. A fact is "promising" if exploring it further is highly likely to lead to the answer or provide crucial context.

Discard facts that are clearly irrelevant, noisy, or are just general information.

OUTPUT FORMAT

Your output **MUST** be a single, valid JSON object with the following structure:

```
{
  "reasoning": "Structured thought process...",
  "pruned_facts": {
    "outgoing": { "P17": ["kept fact labels"], ... },
    "incoming": { "P802": ["kept fact labels"], ... }
  }
}
```

YOUR RESPONSE:

Table 20: Prompt template for the Fact Pruning module. The LLM performs fine-grained filtering of facts to retain only critical evidence or promising exploration leads.

ROLE

You are a research agent resolving an ambiguous search query on Wikipedia.

TASK

Your search for "\${entity}" led to a disambiguation page with multiple possible meanings. You must carefully analyze the full context provided below to choose the single most relevant link from the options.

CONTEXT

- **Original Question:** \${question}
- **Overall Plan (Analysis):** \${analysis}
- **Current Sub-Task (Query):** \${query}
- **Ambiguous Search Query:** "\${entity}"

OPTIONS

\${options_list}

YOUR DECISION

Analyze the options based on the context to select the best fit. Your output must be a single line containing only the exact title of the chosen page from the list. If none of the options seem relevant for answering the original question, output the string "NO_MATCH".

YOUR RESPONSE:

Table 21: Prompt template for the **Disambiguation** step in Adaptive Page Selection. The LLM resolves ambiguous entities by selecting the correct page from disambiguation options based on context.

ROLE

You are a research agent trying to find the right information on Wikipedia after a search query failed.

TASK

Your search for "\${entity}" did not find a matching Wikipedia page. Use the contextual information and the suggestions provided to decide on the best query for your next attempt.

CONTEXT

- **Original Question:** \${question}
- **Overall Plan (Analysis):** \${analysis}
- **Current Sub-Task (Query):** \${query}
- **Failed Search Query:** "\${entity}"

SUGGESTIONS

Below are suggestions from the Wikipedia search API to help you refine your query.

"Did you mean?": This is the API's top recommendation, often correcting a typo or suggesting a more standard page title.

- Suggestion: \${suggestion}

Similar Pages Found: These are pages with titles that are textually similar to your failed query. One of them might be the correct page under a slightly different name.

- Similar Pages: \${search_results_list}

YOUR DECISION

Based on the context and the suggestions, choose the best query for your next search attempt. This could be the "Did you mean?" suggestion, one of the similar pages, or a completely new query you formulate based on the feedback from this failed search. Your output must be a single line containing only the new search query. If none of the suggestions seem useful for answering the original question, output the string "NO_MATCH".

YOUR RESPONSE:

Table 22: Prompt template for the **Search Refinement** step in Adaptive Page Selection. The LLM uses API feedback (corrections and similar pages) to recover from search failures.

ROLE

You are a research agent tasked with answering a complex question by navigating Wikipedia.

TASK

You have been provided with the summary, an infobox table, and the section list of a Wikipedia page relevant to your current query. Your goal is to extract all useful information from the summary and infobox, identify promising sections for deeper investigation, or determine that the page is irrelevant.

CONTEXT

- **Original Question:** \${question}
- **Overall Plan (Analysis):** \${analysis}
- **Current Search Query:** "\${query}"
- **Retrieved Wikipedia Page Title:** "\${page_title}"

AVAILABLE INFORMATION

Page Summary:

\${page_summary}

Summary Infobox:

\${infobox_table}

Page Sections (Top-Level):

\${page_sections}

YOUR DECISION

Based on the information above, what is your next best action? Choose one of the following two options.

1. **If the page seems relevant:**
 - Extract **Direct Information** and **Promising Clues** from the summary/infobox.
 - Identify sections for deeper investigation.
2. **If the page is clearly irrelevant:** Explain why and suggest a better search query.

YOUR RESPONSE:

Table 23: Prompt template for the Global Skimming module. The LLM assesses page relevance, extracts high-level evidence from the summary and infobox, and selects promising sections for detailed reading.

ROLE

You are a meticulous research agent. Your task is to analyze a section of a Wikipedia page that contains both text and tables to extract relevant information for a given query.

TASK

You have been given several retrieved text chunks and a preview of all tables found within a specific Wikipedia section. Your goal is to:

1. Extract **Direct Information** and **Promising Clues** from BOTH the **Retrieved Text Chunks** and the **Table Previews**.
2. Analyze table previews. If a preview is insufficient but promising, select the table for a full read.
3. Provide a single, unified rationale that explains both your information extraction findings and your table selections.

CONTEXT

- **Original Question:** \${question}
- **Current Search Query:** "\${query}"
- **Section Being Investigated:** "\${section_title}"
- **Section Exploration Rationale (during summary reading):** \${exploration_rationale}

AVAILABLE INFORMATION

1. **Retrieved Text Chunks:** (Top \${k} relevant chunks)
\${context_chunks}
2. **Table Previews:** (Previews of tables: \${table_names_in_section})
\${tables_preview}

OUTPUT FORMAT

Rationale: A consolidated explanation covering both text and tables.

Extracted Info: Key info from text chunks and table previews. State "None" if nothing relevant is found.

Selected Tables: List of table names requiring a full read (e.g., ["Awards"]). Empty list [] if none.

YOUR RESPONSE:

Table 24: Prompt template for the **Detailed Reading** phase. The LLM performs joint analysis of retrieved text chunks and table previews to synthesize fine-grained evidence.

ROLE

You are a master AI strategist leading a multi-hop question-answering mission. Your task is to synthesize retrieved information from various sources, evaluate progress against the overall plan, and decide the most logical next step.

CONTEXT

- **Original Question:** \${question}
- **Overall Plan (Analysis):** \${analysis}
- **Notebook (Summary of Known Facts):**
\${notebook}
- **Current Sub-Queries:** \${queries}
- **Core Entities in Sub-Queries:** \${entities}

EVIDENCE

This section contains the information retrieved from different sources for all sub-queries executed in this turn.

`${evidence_blocks}`

YOUR TASK

Carefully review all evidence, and in conjunction with the **Original Question** and your **Overall Plan**, complete the following three steps:

1. **Verbatim Information Extraction:** Meticulously extract all useful information. Your goal is to faithfully transfer potentially relevant information to your 'Extracted Content'. Include:
 - **Direct Facts:** Core facts that directly contribute to answering the original question.
 - **Promising Leads:** New entities or critical factual clues essential for guiding the next step.
 - **[CRITICAL]** Do NOT summarize or rephrase. This is for extraction only.
2. **Think Step-by-Step:** Document your thought process.
 - Explain how the direct facts help answer the question.
 - Discuss the potential value of promising leads and how they might be explored.
 - Identify what key information is still missing.
 - If evidence is useless, explain *why* the current sub-queries failed.
3. **Make a Judgment:** Choose one of the following options:
 - **SUFFICIENT:** Information is adequate to generate a final, complete answer.
 - **INSUFFICIENT_USEFUL:** Valuable clues found, but more information is needed. Continue investigation based on the current findings.
 - **INSUFFICIENT_USELESS:** Information is irrelevant or has led to a dead end. A new strategy is needed.

OUTPUT FORMAT

Thought Process: Your step-by-step thinking process...

Extracted Content: A structured collection of key facts and promising new leads...

Judgment: SUFFICIENT, INSUFFICIENT_USEFUL, or INSUFFICIENT_USELESS.

YOUR RESPONSE:

Table 25: Prompt template for the **Synthesis** module. The LLM aggregates dual-source evidence, extracts key insights, and judges the current progress to determine the next action (answer, continue, or recover). Examples of the content for `${evidence_blocks}` are shown in Tables 26 and 27.

Example of Formatted Candidates

KG Exploration for Mention: "John Cage"

[Entity Linking]

- Linked Entity: John Cage (Q180727)
- Description: American composer and music theorist
- Candidates (top-4 preview):
- John Cage (Q180727): American composer and music theorist
- John Cage (Q5347597): Fictional character from Ally McBeal
- Linking Rationale: The query context discusses "4'33" and "prepared piano", which are signature works of the composer John Cage.

[Retrieved Facts for "John Cage"]

[Attributes]

- date of birth: ['1912-09-05']
- occupation: ['composer', 'philosopher', 'artist']

[Outgoing]

- notable work: ["4'33'", 'Music of Changes', 'Imaginary Landscape No. 4']
- student of: ['Arnold Schoenberg', 'Henry Cowell']

[Incoming]

- influenced by: ['Erik Satie', 'Marcel Duchamp', 'D. T. Suzuki']

[Filtering Summary]

- Relations explored: 5 of 21 total
- Facts kept: 12 of 43 total

[Reasoning]

- Relation Selection: Selected relations regarding works, influences, and personal life relevant to his artistic development.
- Fact Pruning: Kept only major works and key figures like Schoenberg and Cunningham; discarded minor administrative categories.

Table 26: An example of structured KG evidence formatted for the `{evidence_blocks}` in the Synthesis module. It includes the linked entity, verified facts (attributes and relations), and a reasoning trace for the pruning process.

Example of Formatted Candidates

Wikipedia Retrieval Results for entity: "John Cage"

Information Extraction:

- **Page Title:** John Cage
- **All Page Sections:** ['Life', 'Music', 'Visual art...', ...]
- **A. Skimming Summary and Section Selection:**
 - **Rationale:** The summary highlights Cage as a pioneer of indeterminacy and the prepared piano. To understand his specific methods, we need to explore the "Music" section.
 - **Selected Sections for Deeper Analysis:** ['Music']
- **B. Extracted Content:**
 - **From Summary:**

John Cage was a pioneer of indeterminacy in music. Best known for "4'33'", a silent composition. He developed the "prepared piano" and used the "I Ching" as a standard composition tool.
 - **From Sections:**
 - **Section: "Music"**
 - **Rationale for section processing:** Contains detailed descriptions of his composition techniques like chance operations and rhythmic structures.
 - **Extracted from Text:**
 - In 1951, Cage started using the **I Ching** to compose using chance, imitating nature's manner of operation.
 - **Music of Changes** (1951) was the first major work created using this method.
 - **Cheap Imitation** (1969) is a chance-controlled reworking of Erik Satie's Socrate.

Table 27: An example of Text evidence formatted for the `{evidence_blocks}` in the Synthesis module. It summarizes the high-level page content and provides fine-grained details extracted from specific sections relevant to the query.

ROLE

You are a master AI strategist leading a multi-hop question-answering mission. Your task is to plan the next step of the investigation after an information-gathering turn that was useful but insufficient.

CONTEXT

- **Original Question:** \${question}
- **Current Notebook:** \${notebook}
- **Previous Overall Plan:** \${analysis}
- **Previous Sub-Queries:** \${queries}
- **Candidate Entities Pool:** \${candidate_entities_pool}

NEW FINDINGS

- **Thought Process:** \${thought_process}
- **Extracted Content:** \${extracted_content}

YOUR TASK

Plan the next round of investigation to dig deeper and bridge information gaps.

1. **Update Notebook:** Combine current notebook with new findings into a single, coherent summary, retaining all unique and relevant details.
2. **Update Analysis:** Revise the overall plan to reflect new understanding. Explain what the immediate next step should focus on and why.
3. **Plan Next Queries:** Define a new set of sub-queries and corresponding core entities based on the updated analysis.
4. **Manage Candidate Pool:** Review existing leads and identify new promising entities from the new findings. Update the pool by adding new leads and removing promoted ones.

OUTPUT FORMAT

Your output must follow this exact structure:

Thought Process: Your step-by-step reasoning.

Updated Notebook: The comprehensive summary of all known facts.

Updated Analysis: Your revised analysis and plan.

Next Queries: [List of concise and effective search query.]

Next Entities: [List of core entities corresponding to queries]

Updated Candidate Pool: [List of dictionaries, e.g., {"entity": "Name", "reason": "..."}]

YOUR RESPONSE:

Table 28: Prompt template for **Continue Exploration** in the Reflection module. The LLM updates the notebook and analysis based on new findings and plans the next steps to deepen the investigation.

ROLE

You are a master AI strategist leading a multi-hop question-answering mission. Your task is to recover from a failed information-gathering turn where the retrieved evidence was useless.

CONTEXT

- **Original Question:** \${question}
- **Current Notebook:** \${notebook}
- **Candidate Entities Pool:** \${candidate_entities_pool}
- **Interaction History:** \${interaction_history}

FAILED TURN DETAILS

The last exploration turn was deemed unproductive. Here's what went wrong:

- **Previous Overall Plan:** \${analysis}
- **Failed Sub-Queries:** \${queries}
- **Failed Core Entities:** \${entities}
- **Extracted Content:** \${extracted_content}
- **Reasoning for Failure:** \${thought_process}

YOUR TASK

The previous approach hit a dead end. Critically reflect on the history, diagnose the error, and pivot the strategy.

1. **Critical Reflection:** Analyze why the previous approach failed (e.g., flawed plan, wrong entities). Summarize the core strategic error and derive actionable **guiding principles** for the next attempt.
2. **Update Analysis:** Propose a fundamentally new plan that leverages insights from reflection. Consider promoting leads from the Candidate Pool, re-examining the original question for missed keywords, or formulating queries from an entirely different angle.
3. **Plan Next Queries:** Define a new set of queries that represent a clear change in direction from the failed ones.
4. **Manage Candidate Pool:** Review the pool. Decide if existing leads are now high-priority, add any new promising leads discovered incidentally, and generate the updated pool by carrying over unused leads while removing promoted ones.

OUTPUT FORMAT

Your output must follow this exact structure:

Thought Process: Critical reflection on failure and reasoning for the new plan.

Updated Analysis: The revised analysis and new strategic direction.

Next Queries: [List of concise and effective search query.]

Next Entities: [List of core entities corresponding to queries.]

Updated Candidate Pool: [List of updated candidate dictionaries, e.g., {"entity": "Name", "reason": "..."}]

YOUR RESPONSE:

Table 29: Prompt template for **Strategy Adjustment** in the Reflection module. The LLM diagnoses the failure using interaction history and pivots the strategy by generating entirely new queries or backtracking to candidate entities.